Hello World

OPEN SOURCE TEAM TAIWAN

2025

Exhibitor Guide

數位發展部 數位產業署
Administration for Digital Industries, moda

Open. Contribute. Lead.

Built on Taiwan's
proven hardware strength,

we are growing
an open software ecosystem
for the AI era.

Companies create.
Communities collaborate.
Developers leave their footprints
on tomorrow's open source.

15 Projects / 4 Categories / One Mission

Software-led
OSS Business

Hardware-led
OSS Innovation

Community &
Academic OSS

Taiwanese Footprints in
World-Class OSS

Pavilion Talks Agenda | Scan for details

Contents /

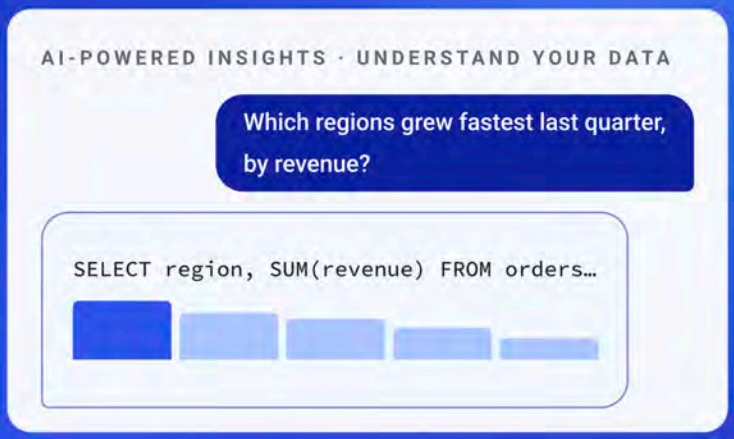
WrenAI

JUN 2 – 5, 2026

Generative BI, reimagined for the agentic era.

Turn any AI Agents into world-class data analysts through the open context layer that gives AI agents grounded, governed SQL across 20+ data sources, that helps you build GenBI, dashboards, and agentic analytics.

★ #1 OPEN-SOURCE GENBI GitHub ★ 15,000+



PRODUCT HIGHLIGHTS

01 ZERO LEARNING CURVE

Just talk to your data

Connect a database. Ask in plain language. Get the answer. No SQL. No BI training. No modeling language to learn.

02 REUSABLE BY EVERY HUMAN AND AGENT

Knowledge as assets

Insights, business context, analytical frameworks, metric logic — captured by Wren AI, not lost in someone's head. When people leave, the knowledge stays.

03 VIBE-CODED GENBI APPS

Question to dashboard, one prompt

Describe what you need; the Agent builds it. Governed by your semantic layer, versioned in Git.

04 BEYOND LLM GUARDRAILS

Unified data policy engine, governs AI at the data layer

Enforce data access and governance through a centralized policy engine. Wren AI secures data directly at the semantic and data layer across every model, agent, and dashboard.

THREE WAYS TEAMS USE WREN AI

01 REPLACE

Replace traditional BI

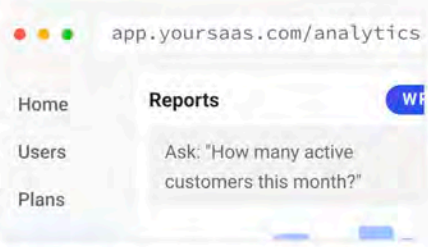
Healthcare · Finance · Manufacturing

Retire traditional BI with a conversational interface — same warehouse, same governance, zero seat-licence rot.

02 EMBED

Embedded GenBI for your platform

SaaS vendors · Platform builders

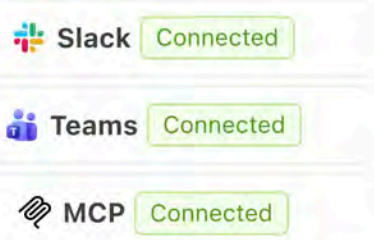


Embed Wren AI into your product; ship Agent-powered analytics to your users — your branding, our infrastructure.

03 BUILD

Bring analytics into Slack & Teams

AI Engineering · Platform Teams



Use Wren AI's API to drop an agent into the apps your team already lives in.

OUTCOMES

10×

faster insights

Transform raw data into actionable business insights with self-service, AI-driven analytics.

90%

less SQL writing

Perform advanced analytics and statistical analysis on your datasets, with no SQL fluency required.

20+

hours saved per month

Automate recurring reporting and empower teams to answer data questions instantly.

WEB getwren.ai GITHUB github.com/Canner/WrenAI EMAIL contact@getwren.ai

FEATURE HIGHLIGHTS

Agentic Mode at the core. Skills, Memory, Architecture, GenBI Apps on top.

The semantic layer is the foundation. Four pillars compound on it.

01 AGENTIC MODE

Not just chat.

Sandboxed multi-step reasoning. The Agent uses tools and writes scripts inside an isolated environment — query, chart, extract knowledge from PDFs, build dashboards, save skills.

02 SKILLS & MEMORY

Reusable workflows · context that compounds.

Skills are SOPs for your Agent — consistent, high-quality output. Memory turns past interactions, preferences and corrections into institutional knowledge.

03 AGENT-FRIENDLY ARCHITECTURE

Git native + File System.

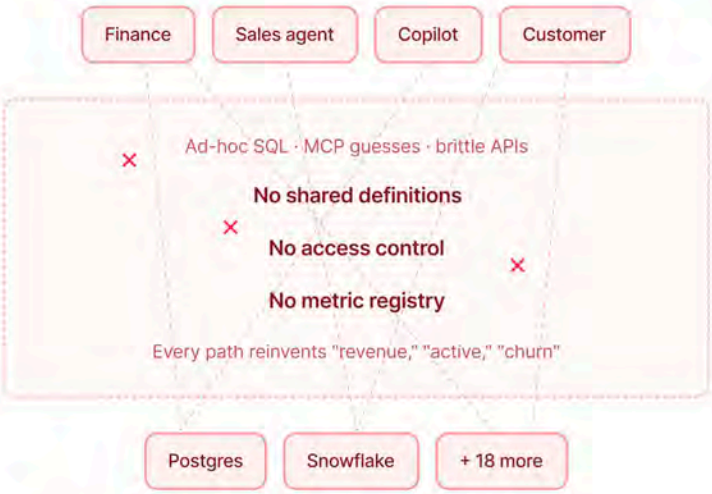
Skills, Memory, Semantic Model and Instructions are all files — version-controlled, branched, PR-reviewed, rolled back. Agents read and write directly.

04 GENBI APPS

Vibe-coded dashboards in one prompt.

Describe what you need; one click renders a polished, semantic-layer-governed dashboard. No SaaS click-through to pick fields.

BEFORE & AFTER



CUSTOMER STORIES

Wren AI enables natural language data interaction across 10+ databases without ETL or migration. Integrated with Phison's aiDAPTIV+ architecture, it delivers up to an 80% query hit rate and secure, on-prem AI operations—accelerating adoption with lower costs and faster deployment.

Wei, Lin
Phison CTO (8299.TWO)

Wren AI powers our SaaS product, DemandSense, enabling clients to naturally ask questions, generate insights, and create real-time charts or dashboards. It's a game-changer for making analytics accessible without technical expertise.

Anna Khamitova
CTO, Impactable (United States)

Wren AI transformed our decision-making. By enabling instant, natural language insights without complex reporting, our decision speed improved by over 50%. Data shifted from an 'expert tool' to a 'company-wide asset,' revolutionizing our data culture.

Shasta Ho
CEO, Nextlink (TPEX:6997)

GET STARTED

Free OSS · Try Commercial

WEB getwren.ai GITHUB github.com/Canner/WrenAI EMAIL contact@getwren.ai

OPEN SOURCE COMMERCIAL

Empowering the Agentic AI Era

Enterprise Open Source AI Gateway Infrastructure

digiRunner Open Source | COMPUTEX 2026

“If you want to go fast, go alone; if you want to go far, go together.”

Enforce AI Governance

Establish Absolute Trust

Bridge intelligence

The Threat Landscape of Agentic AI

Autonomous AI brings new power, but critical roadblocks remain:

Cost Sprawl

Data Vulnerability

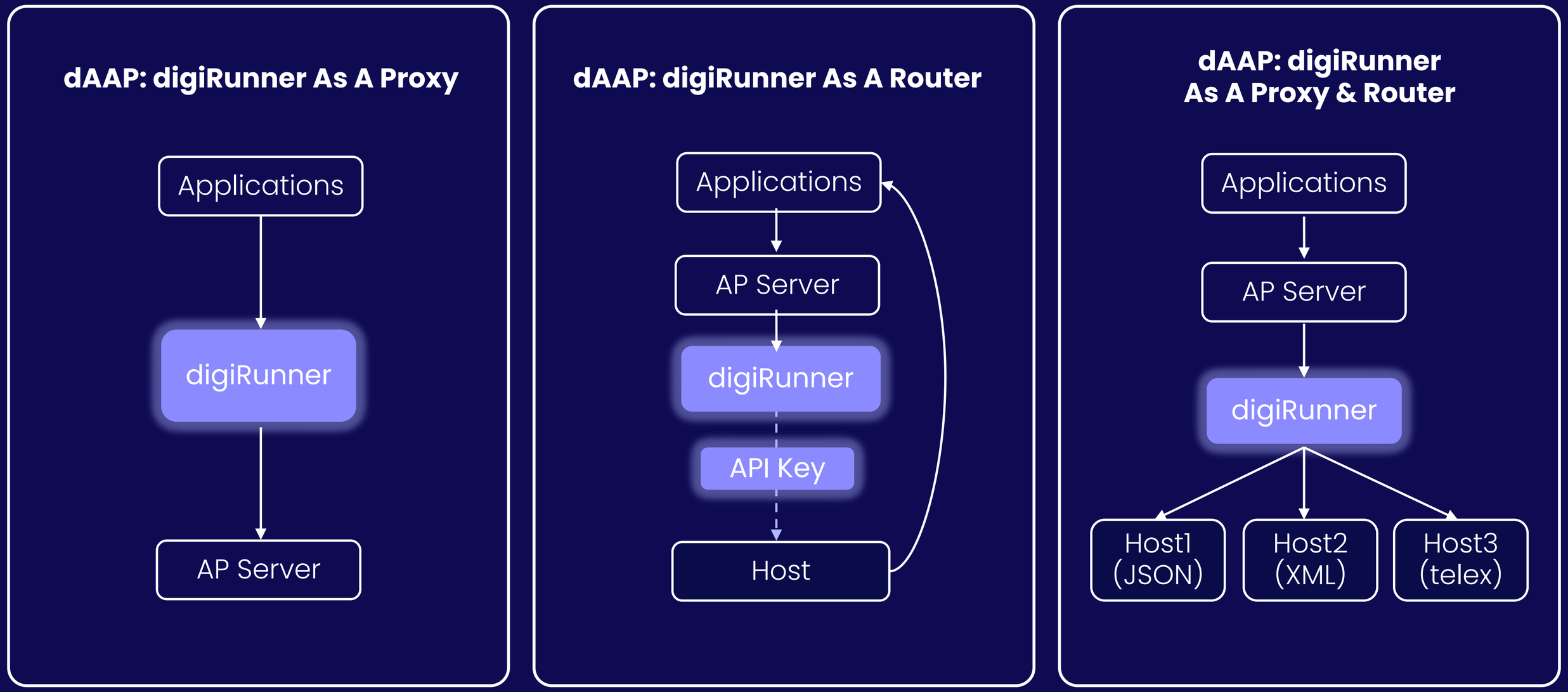
Rigid Integration

Traffic Governance

Zero-Trust Security

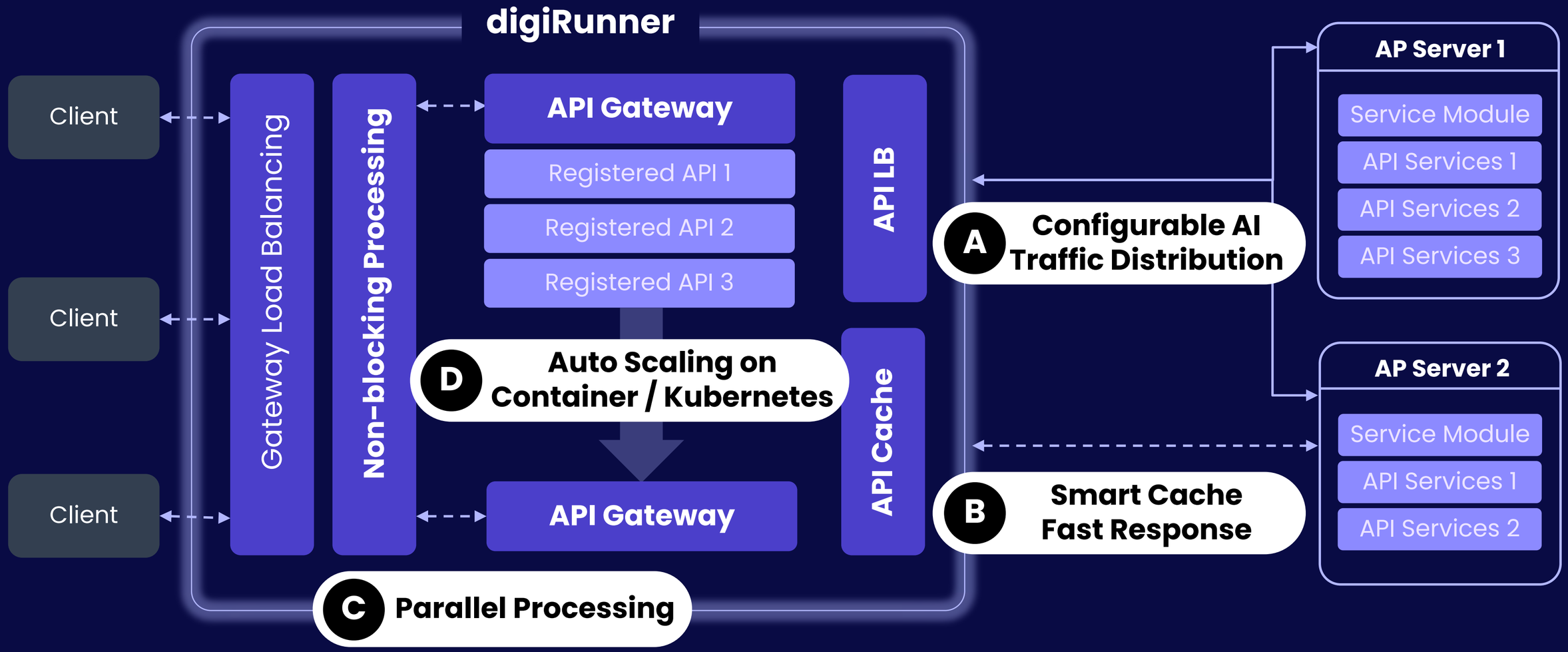
Standardized Integration

Architectural Topologies: Flexible API Routing

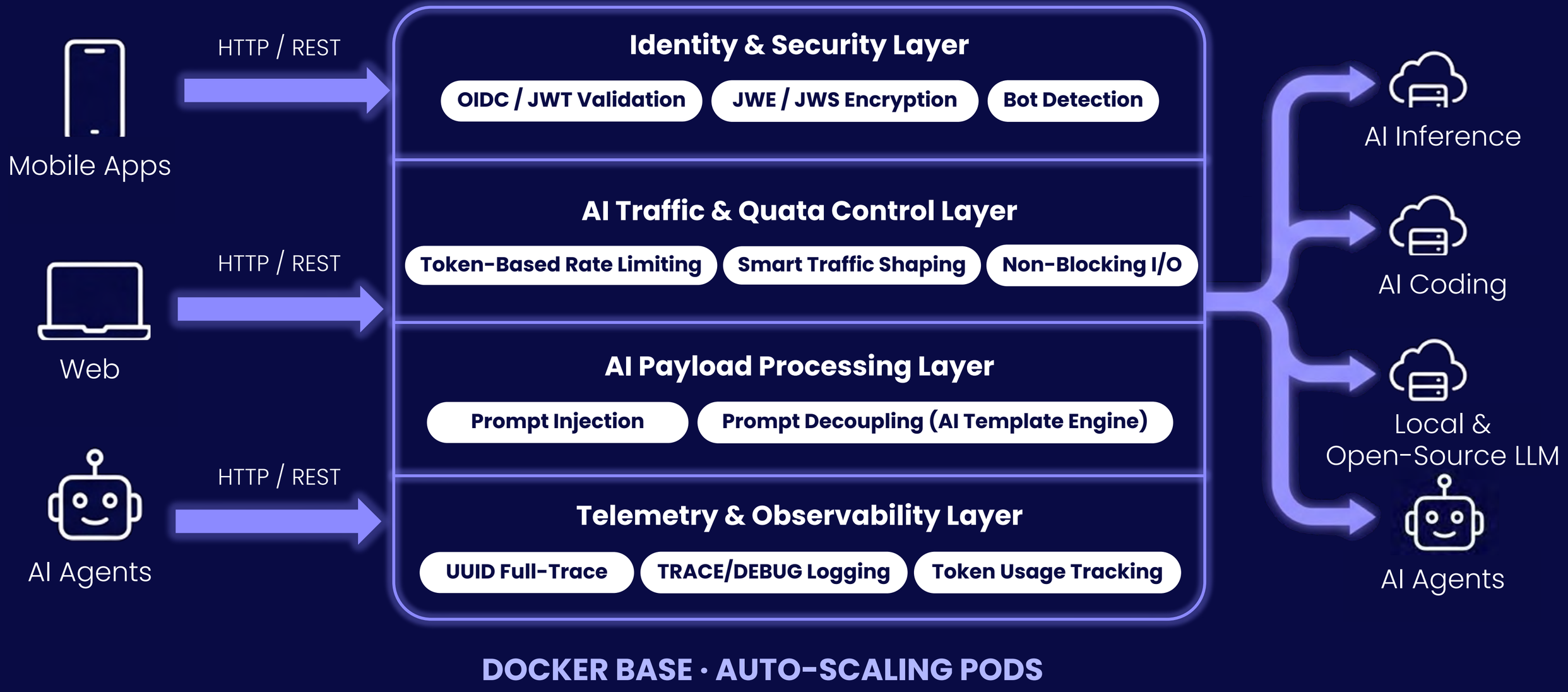


High Available Framework for Mission Critical Services

Deliver flexible API handling capability targeting Business Continuity Plan in enterprises



The AI-Native Control Plane Architecture



DOCKER BASE · AUTO-SCALING PODS

Enterprise Application Blueprints

Bridging AI to Enterprise Reality across Highly Regulated Industries

Finance

- Real-Time Fraud Detection
- Automated KYC/AML

Healthcare

- Seamless FHIR Integration
- Dynamic Privacy Shield

Public Sector

- Legacy API Unification
- Cross-Agency Data Sharing

A Dual-Track Open Source Global Strategy

Middle East (UAE) | Open Finance

UAE Central Bank mandates: 20+ institutions

High-ROI platform licensing

South Asia (Pakistan) | Talent Hub

Massive IT-savvy demographic

Strategic stepping stone to NA/EU markets

Global Monetization Strategy: The Open Core Model

Core is Free. Revenue from Enterprise Support, Training, and Premium Features.

Open Source in Action: Campus Co-creation

Building Secure Enterprise AI with University Innovators

Awareness

digiRunner Open Source x Dify

Chat UI ⟶ digiRunner API ⟶ AI (Dify)

- Collaboration Project: digiRunner Open Source x Dify
- Open Innovation: Co-Creation & Knowledge exchange
- On-campus Engagement & Interaction
- Community Content & Amplification
- Outcome & Public Exposure

University Academic Plan

- Professor Research Projects
- Department Courses

Google Developer Groups (GDG)

- Events
- Discord
- LinkedIn Community

GDG on Campus

- **NTPU**
- **NCCU**
- **NCU**
- **NCHU**
- **TMU**

2026

OpenTPI

digiRunner Open Source Project

OpenTPI

Scan to connect!

On mobile?
Click to explore!

CubeCOS

Enterprise Open Source Cloud Operating Platform for Modern Virtualization Workloads.

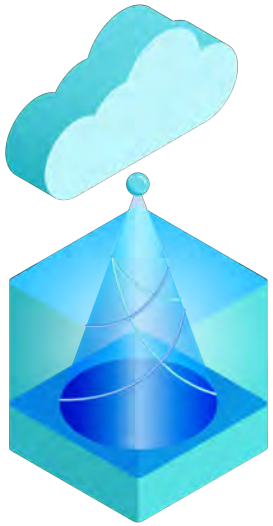
Turn commodity x86 hardware into a complete cloud platform without the enterprise price tag.

cubeCOS by Bigstack

Modern Infrastructure Challenges

VMware Alternative

To replace VMware and avoid vendor lock-in.



Carrier-Grade Cloud

That meets the scalability, maintainability and reliability standards of the telco industry.

AI Infrastructure

Create and manage multi-tenant GPU clusters with tenant-level isolation on your infrastructure on a large scale.

cubeCOS by Bigstack

Delivering Unified Infrastructure Experience

| VMaaS | KaaS | Integrated Multi-Factor Authentication (MFA) and Single Sign-On (SSO) |

| Disk & Image Management | Dynamic User-Specific Firewall Rule Generation | Multi-Tenant Networking | Network Load Balancing |

Single-image deployment. Scale from one to multiple nodes. Managed open-source lifecycle.

IaaS/PaaS delivery. Multi-tenancy isolation. Hybrid cloud integration.

Simplified Infrastructure

Self-Service User Experience

Cube Cloud Operating System

cubeCOS

cubeCOS by Bigstack

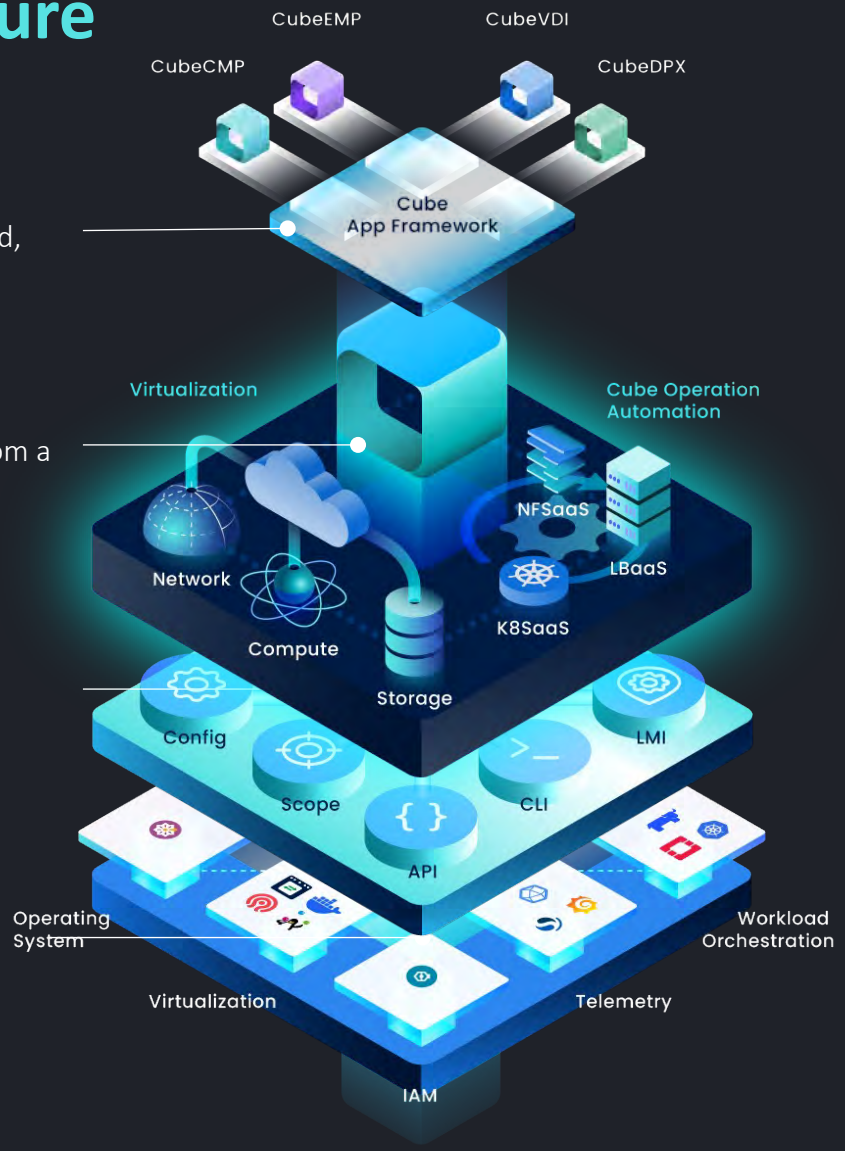
Composable Architecture

Cube App Framework
Enterprise add-on modules for remote work, hybrid cloud, and data protection requirements.

Cube Cloud Operating System (COS)
VM, Kubernetes and modern storage are all managed from a single control plane.

Bigstack HEX bedrock©
Automating cluster deployment, operation and maintenance simplifies open-source management.

Enterprise Ready Open Source
Full community transparency with the stability and support production demands.

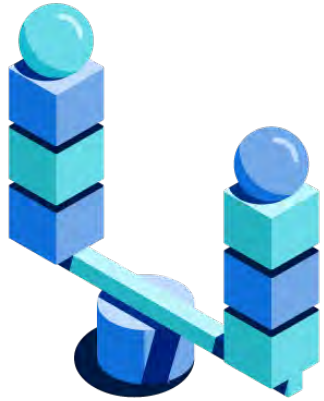


- Extensible Modules
- One-Platform Full Stack
- Self-Service by Design
- Carrier-Grade Reliability
- Apache 2.0 License

Architecture for Modern Infrastructure Success

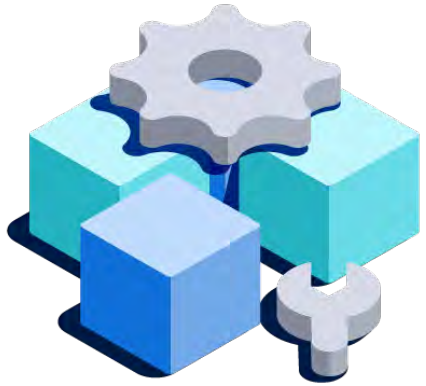
Deployability

Single-image deployment compresses cloud rollout from months to weeks. Consistent, repeatable deployments across every environment



Stability

A distributed architecture that removes single points of failure by design. Workloads continue uninterrupted when individual nodes go down



Maintainability

One-click update and upgrade simplifies the full open source lifecycle. Guided workflows replace manual, error-prone patching processes

Customer Stores

National Class A Data Center

Supplying AI inference services to research and education users.

Highlights

Seamless Data Flow

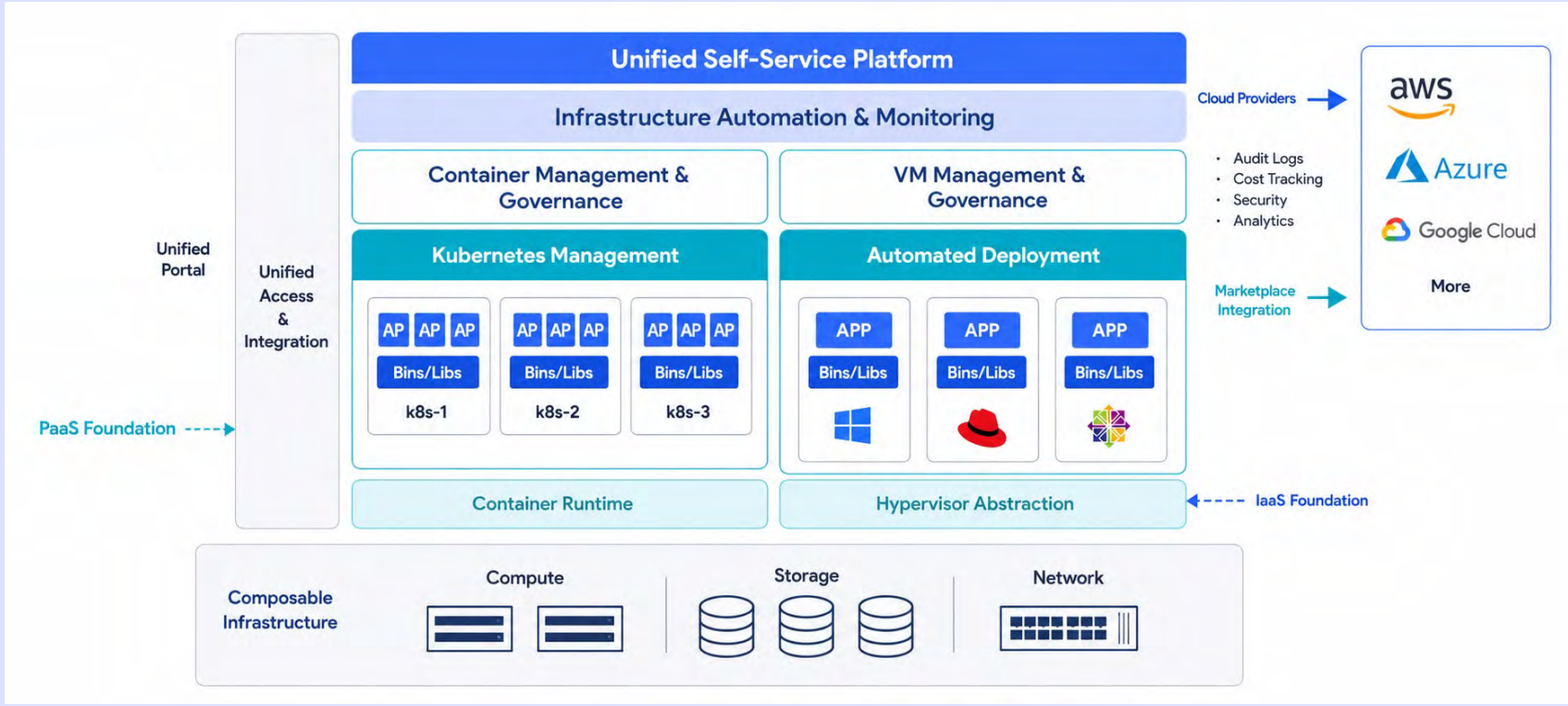
A unified management layer eliminates cross-system bottlenecks, giving operators a single, consistent view of all workloads and resources.

Secure Multi-Tenant Isolation

CubeCOS enforces strict data isolation between tenants while enabling coordinated processing across shared infrastructure.

Standardized Infrastructure

Consolidating x86 hardware under a unified virtualization layer cuts costs and improves resource utilization across the board.



Telco Data Center Cluster

Powering telecommunication cloud with performance, ease of management, and telco scale.

Highlights

Flexible Management Design

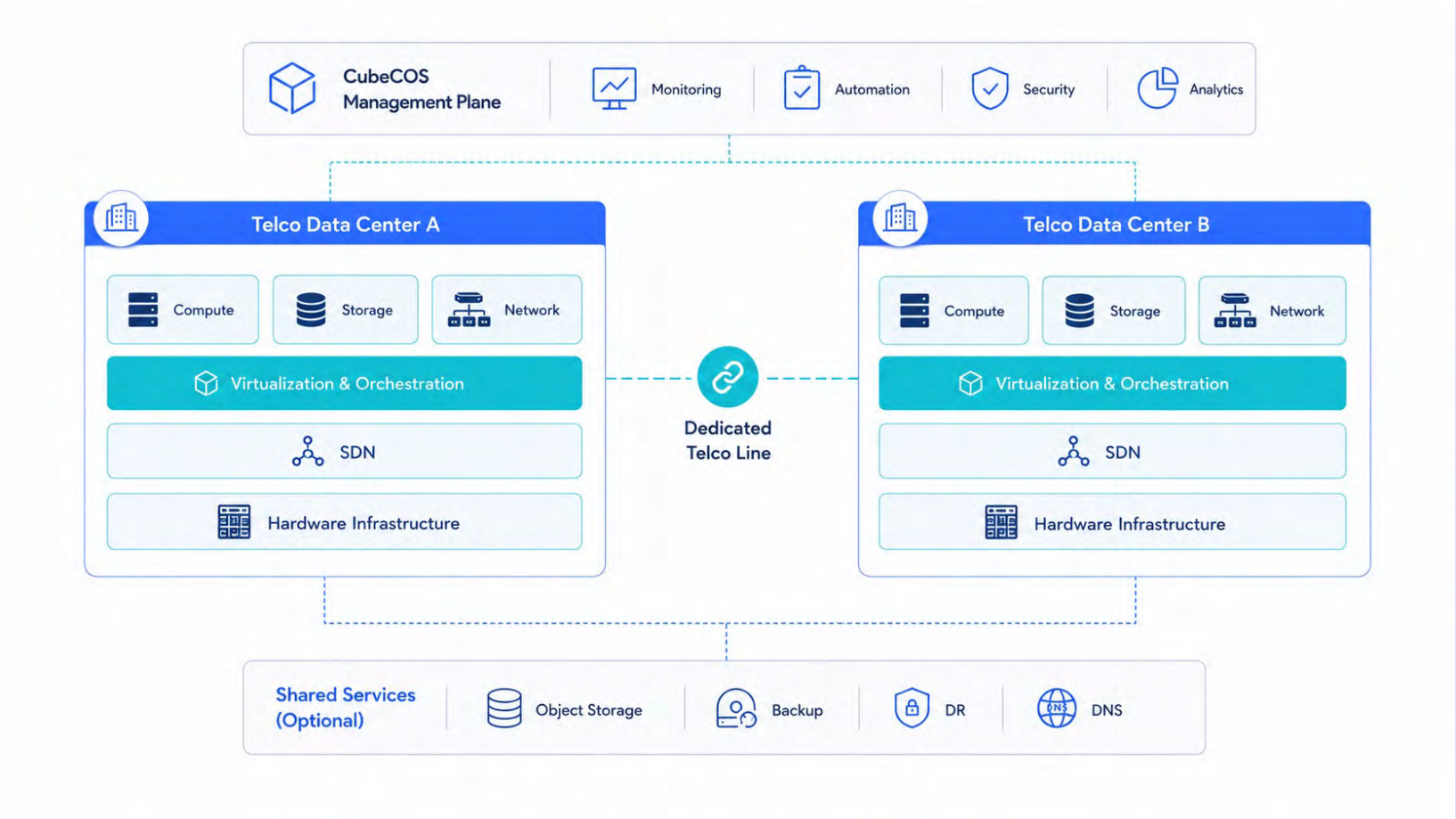
Manage multiple clusters through a unified management plane with support for shared portals across clusters or dedicated portals per site.

Multi-tenant NFV

Multitenant NFV: one platform, many services empowering telcos to deliver isolated, scalable network functions to every customer without compromise.

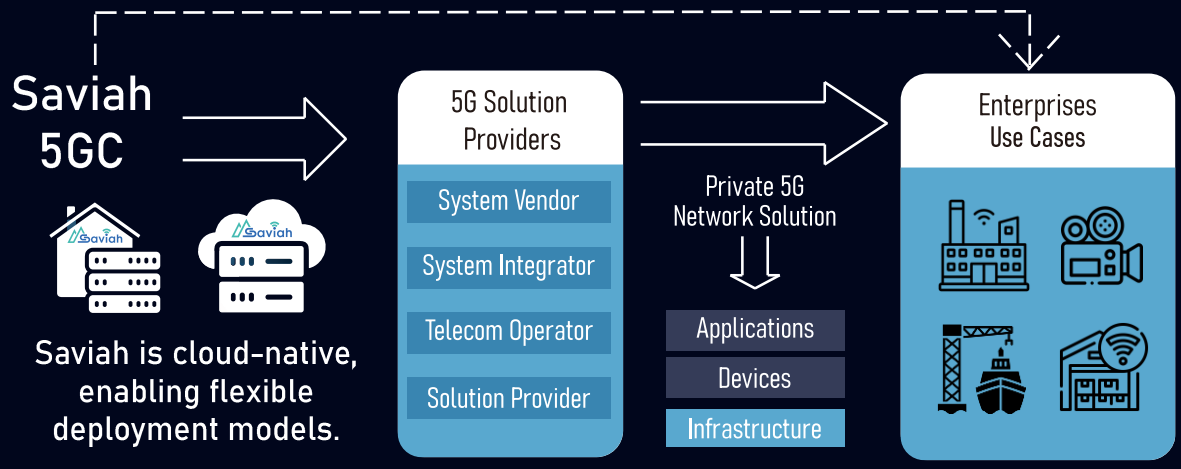
Sovereign Architecture

Maintain full data sovereignty across distributed telco infrastructure with a unified management plane that adapts to your deployment model.



Saviah is an innovative 5G core network software provider. Compliant with 3GPP R15, 16 and 17, Saviah 5GC enables **enterprises** and **solution providers** to deploy highly reliable, secure and scalable private 5G network solutions for diverse industrial use cases.

Saviah provides 5GC Software as a Service and creates business value using **a B2B2X model.**



Professional & Responsive

Pre- and post-sales support

Proven Reliability

Carrier-grade stability for mission-critical private 5G

Seamless Integration

Supports Open API & RESTful APIs for easy integration

Advanced Capabilities

Supports a wide range of industrial and enterprise use cases

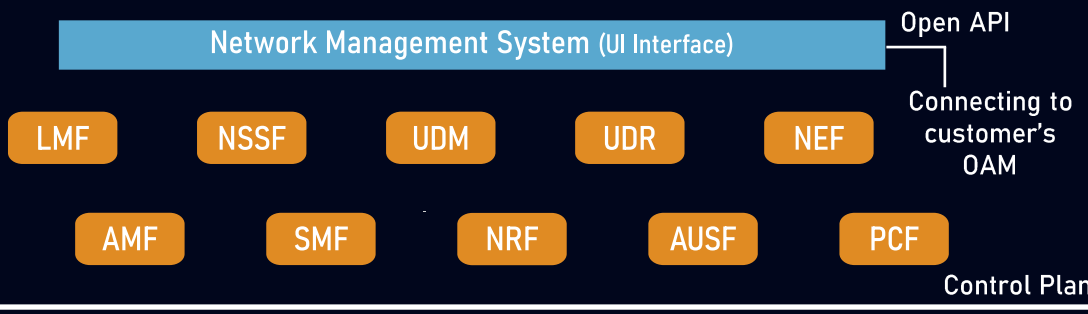
Saviah 5GC has enabled **50+ private 5G** deployments across industries.

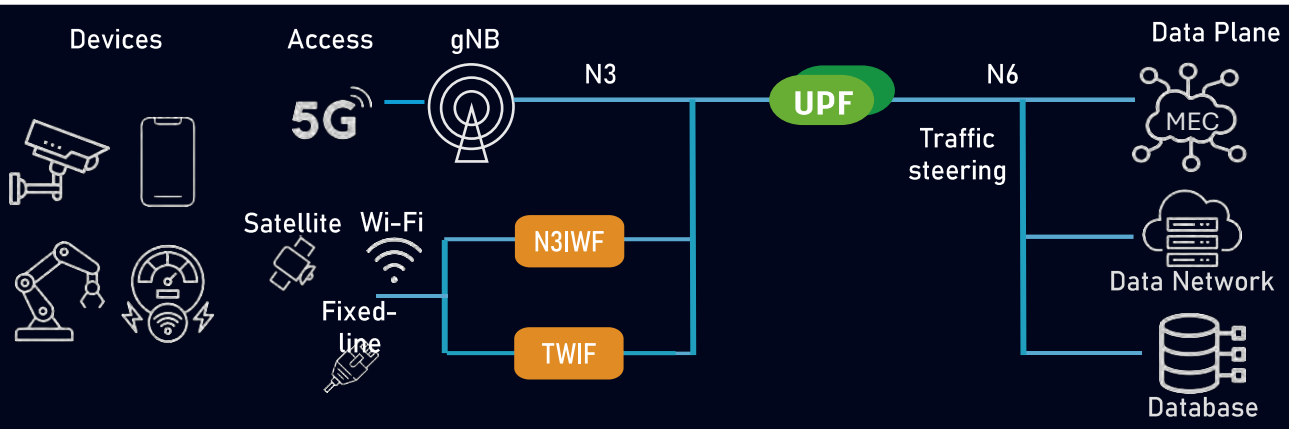
Website

Saviah
Connecting Intelligence

LinkedIn

Saviah 5GC Network Functions





Saviah 5GC is a fully containerized, cloud-native 5G core – secure, reliable, and built with **advanced capabilities** for diverse private 5G and mission-critical networks.

NEF **High Availability** **5G LAN** **TSN**

NEF securely **exposes network capabilities** to **authorized third-party applications** through standardized APIs, enabling apps to request network services (e.g., QoS, session information, event notifications) without direct access to internal network functions.

Use Cases of NEF

Dynamic Configuration

Applications can dynamically request and adjust QoS for different service flows via standard interfaces, **without** restarting the 5GC.

Location- & event-driven services

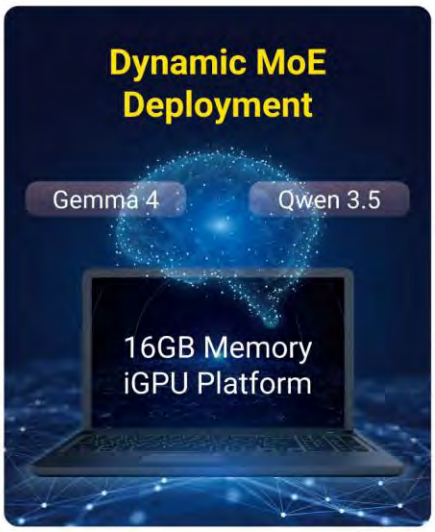
Applications subscribe to network events and trigger actions with location logic (e.g., forklift geofencing, worker safety alerts).

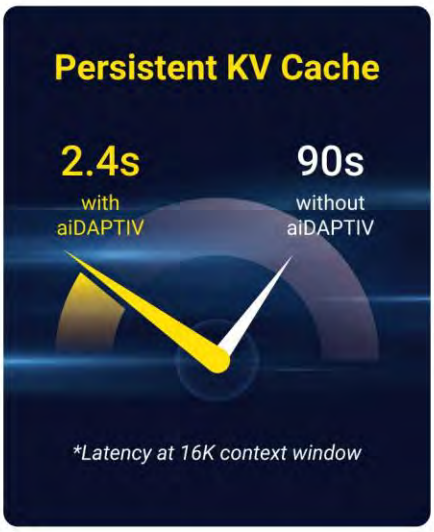
High availability and **geo-redundancy** ensure service continuity by enabling failover to a standby UPF or a backup site when a network function, link, or site outage occurs—minimizing interruption and preserving the session/IP address.

Phison Hybrid Router for OpenClaw

Local Control. Lower Cost

Phison Hybrid Router delivers high-performance edge AI for OpenClaw.
Leveraging Pascari aiDAPTIV, it runs large-scale models locally—slashing cloud costs, securing data privacy, and optimizing operational efficiency.





intel. ASUS msi acer GMKtec ASRock Industrial

What's Pascari aiDAPTIV™?

Pascari aiDAPTIV™ is a purpose-built solution that combines aiDAPTIV Cache Memory and aiDAPTIV Memory Management Middleware to help memory-intensive AI workloads run on local systems by extending effective memory.

aiDAPTIV Cache Memory

A purpose-built SSD optimized for AI workloads. This acts as an additional memory tier to extend total memory for AI.

aiDAPTIV Memory Management Middleware

The software layer that orchestrates data across GPU memory, system memory, and aiDAPTIV Cache Memory.

Benefits

• For training

aiDAPTIV helps teams keep workloads on-prem for better privacy and data sovereignty, while reducing the need for larger and more expensive GPU infrastructure.

• For inference

Run more complex, memory-hungry workloads locally, including larger models, longer context, and agentic AI workflows.

Private AI Processing

Simple to Use and Deploy

Budget-Friendly

MEDIATEK Research

COMPUTEX 2026

Breeze 3: Taiwan-Focused AI for Everyone.

MediaTek Research builds AI models for Traditional Chinese users. Breeze 3 enables Taiwanese Hokkien understanding and speech, enhances communication, and strengthens digital safety in a localized ecosystem.

SPEECH RECOGNITION

Breeze ASR

Whisper-class Taiwanese Hokkien speech recognition. Sub-second inference on local GPU with BFloat16 precision.

CONTENT SAFETY

Breeze Guard

Traditional Chinese safety classifier covering scam, health misinfo, political manipulation, and more. 6 safety categories.

SPEECH SYNTHESIS

Breeze TTS

Natural Taiwanese Hokkien text-to-speech powered by CozyVoice v2 architecture. High-fidelity voice generation. *Available via API; not open-sourced.

<1s	100%	6	Open*
INFERENCE LATENCY	ON-DEVICE PRIVACY	SAFETY CATEGORIES	ASR & GUARD WEIGHTS

- Runs entirely on local GPU — no cloud dependency, no data leaks
- Single HTTP endpoint — POST audio, get JSON transcription
- torch.compile graph caching for sub-second warm inference
- Optimized for Taiwanese Hokkien (台語) with native cultural context
- Taiwan Safety Benchmark (TSBench) for localized content moderation
- Open-source model weights (ASR & Guard) and technical reports on ArXiv

封面:

標題
中央研究院｜COMPUTEX 2026 展館專案介紹
Academia Sinica｜COMPUTEX 2026 Featured Projects

TAIDE - 推動臺灣可信任生成式AI發展計畫
TAIDE - Trustworthy AI Dialog Engine

簡要介紹
TAIDE 是國家級生成式 AI 發展計畫，打造深度理解正體中文、臺灣語境與在地文化的可信任大型語言模型。

TAIDE is a national-level generative AI initiative, aiming to develop trustworthy large language models that deeply understand Traditional Chinese, Taiwan's linguistic context, and local culture.

內容:

專案介紹
TAIDE（Trustworthy AI Dialogue Engine）自 2023 年啟動，匯聚臺灣人工智慧研究能量，致力於打造深度理解正體中文、臺灣語境與在地文化的大型語言模型。

本計畫以「可信任、合法授權、安全對齊與在地化應用」為核心，透過高品質資料訓練與持續模型優化，降低語言偏誤與不當生成風險，提供政府、產業與學術界可運用的基礎 AI 資源。

本次參展將展示 TAIDE 自主研發成果，呈現其在在地語言理解、安全設計與實務應用上的優勢，並期望透過產業交流與使用者回饋，促進合作機會與技術落地。未來 TAIDE 將持續強化推理能力、多模態整合與檢索技術，推動臺灣 AI 生態系的穩健發展。

Project Overview

TAIDE, short for **Trustworthy AI Dialogue Engine**, is a national-level AI development project jointly promoted by Academia Sinica and the National Science and Technology Council. Launched in 2023, the project brings together Taiwan's AI research expertise to develop large language models that deeply understand Traditional Chinese, Taiwan's linguistic context, and local culture.

Centered on trustworthiness, legally licensed data, safety alignment, and localized applications,

TAIDE uses high-quality training data and continuous model optimization to reduce linguistic bias and the risk of inappropriate generation. It provides a foundational AI resource for government, industry, and academia.

At COMPUTEX 2026, TAIDE will showcase its independently developed model achievements, highlighting its strengths in local language understanding, safety design, and practical applications. Through industry exchanges and user feedback, the project aims to promote collaboration opportunities and support real-world technology adoption.

專案亮點 Key Highlights

1. 深度理解正體中文與臺灣語境

TAIDE 強調對正體中文、臺灣語境及在地文化的理解，能更貼近臺灣使用者、政府機關、產業與學術研究場域的需求。
Traditional Chinese and Local Context Understanding
TAIDE is designed to understand Traditional Chinese and Taiwan-specific linguistic and cultural contexts, making it well suited for local government, industry, academic, and public-service applications.

2. 採用合法授權與高品質資料

TAIDE 重視資料來源的合法性與品質，透過合法授權資料進行訓練，降低資料使用風險，並建立更可信任的 AI 基礎。
Legally Authorized and High-Quality Data
TAIDE emphasizes the use of legally authorized, high-quality data, helping reduce data governance risks and strengthen trust in AI development.

3. 重視安全性與價值對齊

TAIDE 以安全對齊與可信任為核心，持續降低語言偏誤、不當生成與應用風險，提升模型在實務場域中的穩定性與可靠性。
Safety and Value Alignment
TAIDE focuses on safety alignment and trustworthiness, continuously reducing linguistic bias, inappropriate generation, and deployment risks.

4. 支援開放授權與商業應用彈性

TAIDE 具備開放授權與商業應用彈性，有助於政府、產業與學術界導入應用，促進臺灣生成式 AI 生態系發展。
Open Licensing and Commercial Flexibility
TAIDE’s open licensing and commercial flexibility enable broader adoption across government,

industry, and academia.

5. 持續強化推理、多模態與檢索技術

未來 TAIDE 將持續提升推理能力，拓展多模態與檢索技術，並強化公務相關應用，同時依據全球 AI 趨勢與使用者回饋進行迭代更新。
Advancing Reasoning, Multimodal, and Retrieval Technologies
TAIDE will continue enhancing reasoning capabilities, expanding multimodal and retrieval technologies, and improving government-related applications through iterative updates based on global AI trends and user feedback.

專案連結 Project Links

TAIDE 官方網站
https://taide.tw/

TAIDE 開源平台下載區
可於現場掃描 QR Code 查看相關模型、資源與下載資訊。

聯絡方式Contact

參展單位 Exhibitor
中央研究院 Academia Sinica

聯絡人 Contact Person
黃瀚萱
中央研究院資訊科學研究所副研究員
Associate Research Fellow, Institute of Information Science, Academia Sinica
hhhuang@iis.sinica.edu.tw

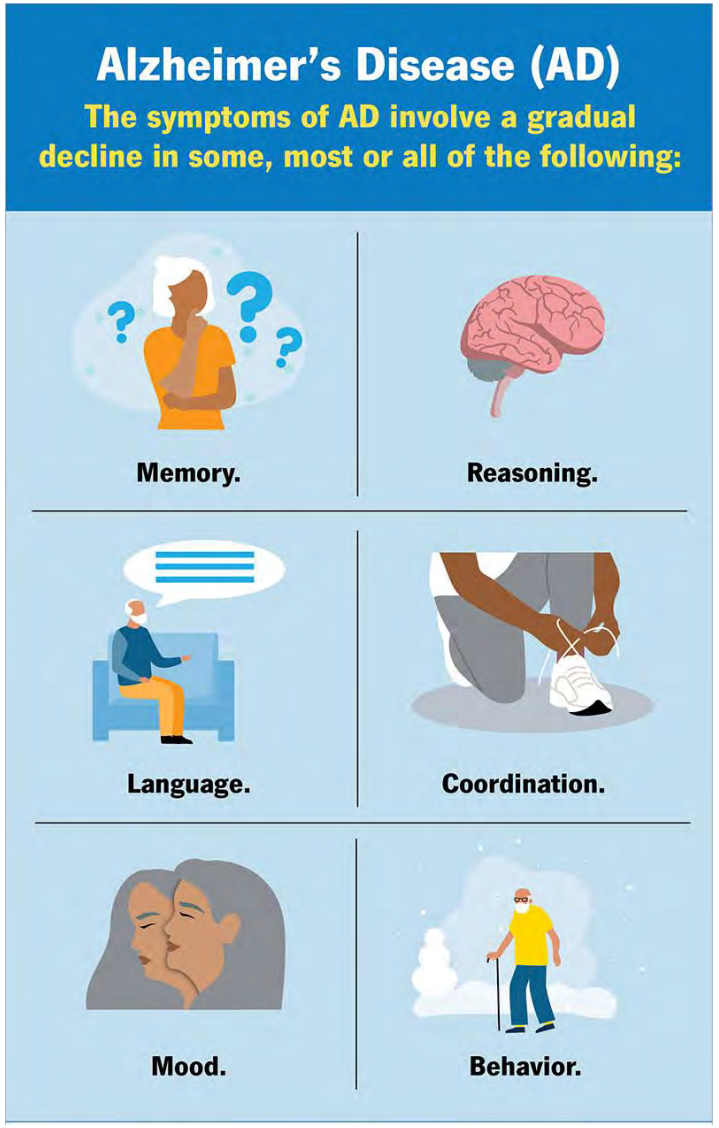
Deep Learning-Based Animal Behavior Analysis: Insights from Mouse Models of Alzheimer's Disease

Papers : https://arxiv.org/abs/2602.09518
https://arxiv.org/pdf/2508.05138

Project page : https://github.com/franktpmvu/Universal-Action-Space.git

The Stages of Alzheimer's Disease (AD) in Humans

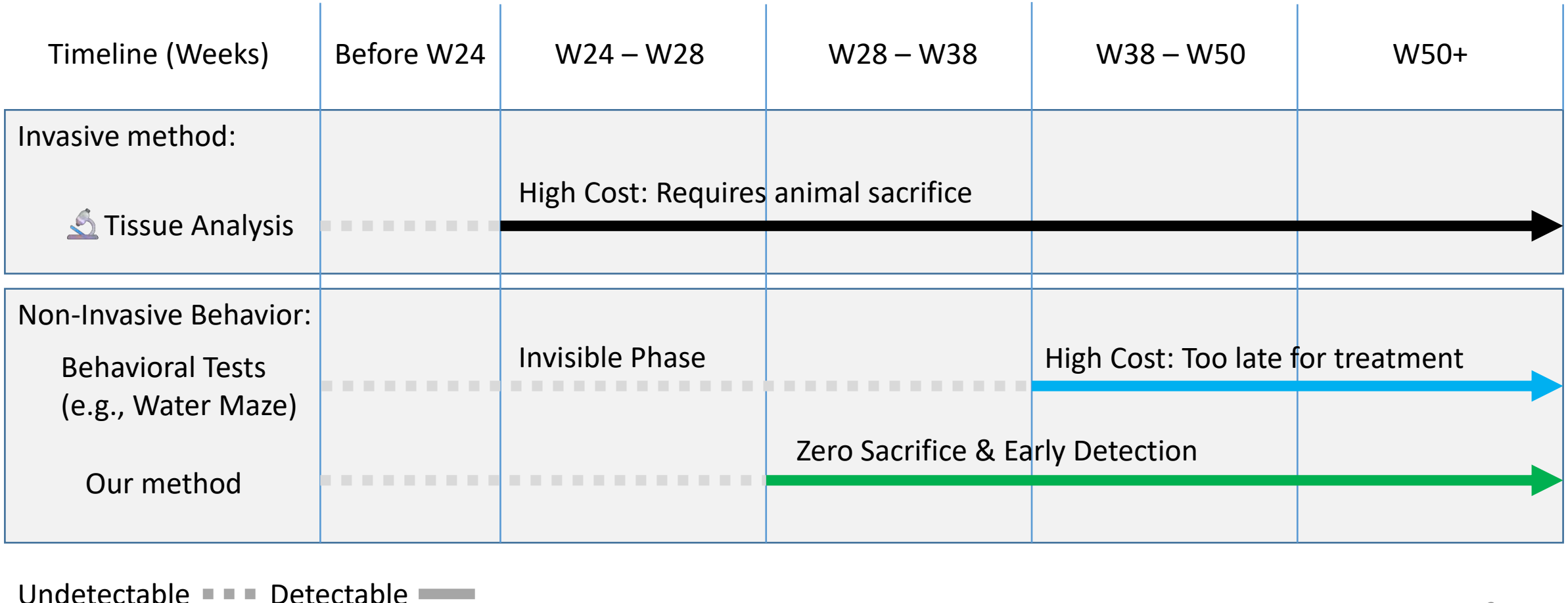
- **Early stage:** short-term memory decline, word-finding difficulties, mild disorientation, anxiety/depression.
- **Middle stage:** getting lost in familiar places, difficulty speaking and understanding, increasing dependence in daily activities.
- **Late stage:** failure to recognize family members, severe language impairment, motor function decline, fully dependent on care.



https://my.clevelandclinic.org/health/diseases/9164-alzheimers-disease

The Difficulty of Early Detection in Alzheimer's Disease in Mouse Models

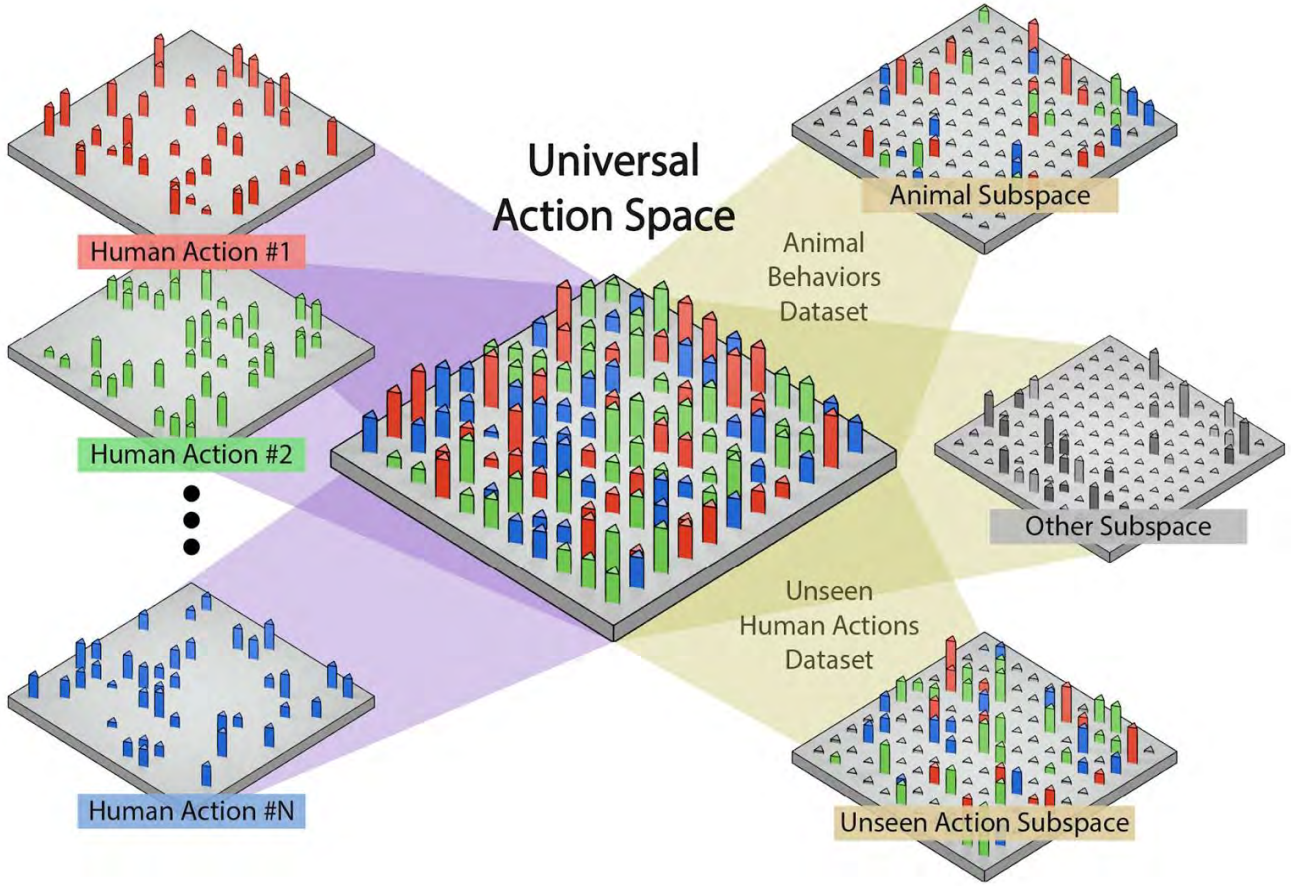
- A critical time lag exists between invasive biological markers and observable behavioral symptoms.
- Subtle and gradual behavioral changes that are hard to quantify



AI-Powered Behavioral Phenotyping

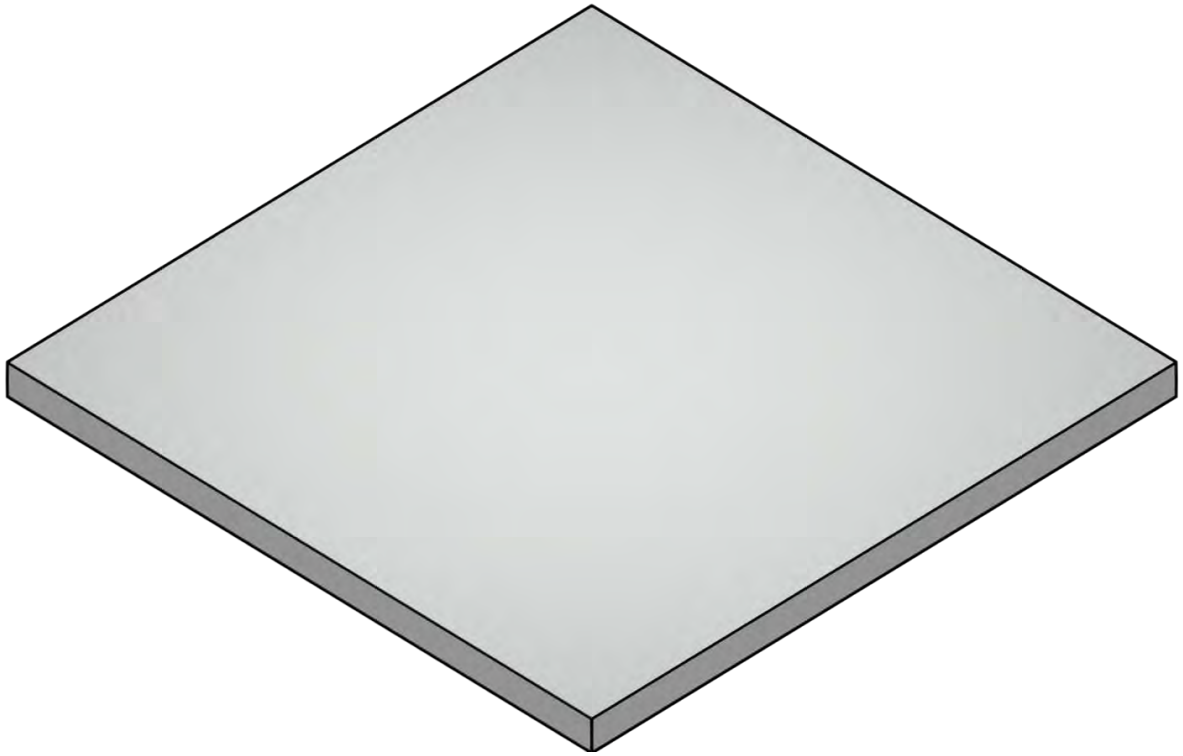
- Decoding Alzheimer’s symptoms within the "Invisible Phase" using periodic 5-minute multi-view snapshots.

- **5-Min Bi-weekly Snapshots**
- **Dual-View Integration**
- **Dataset:** 128 Standardized Videos (8 AD / 8 Control)



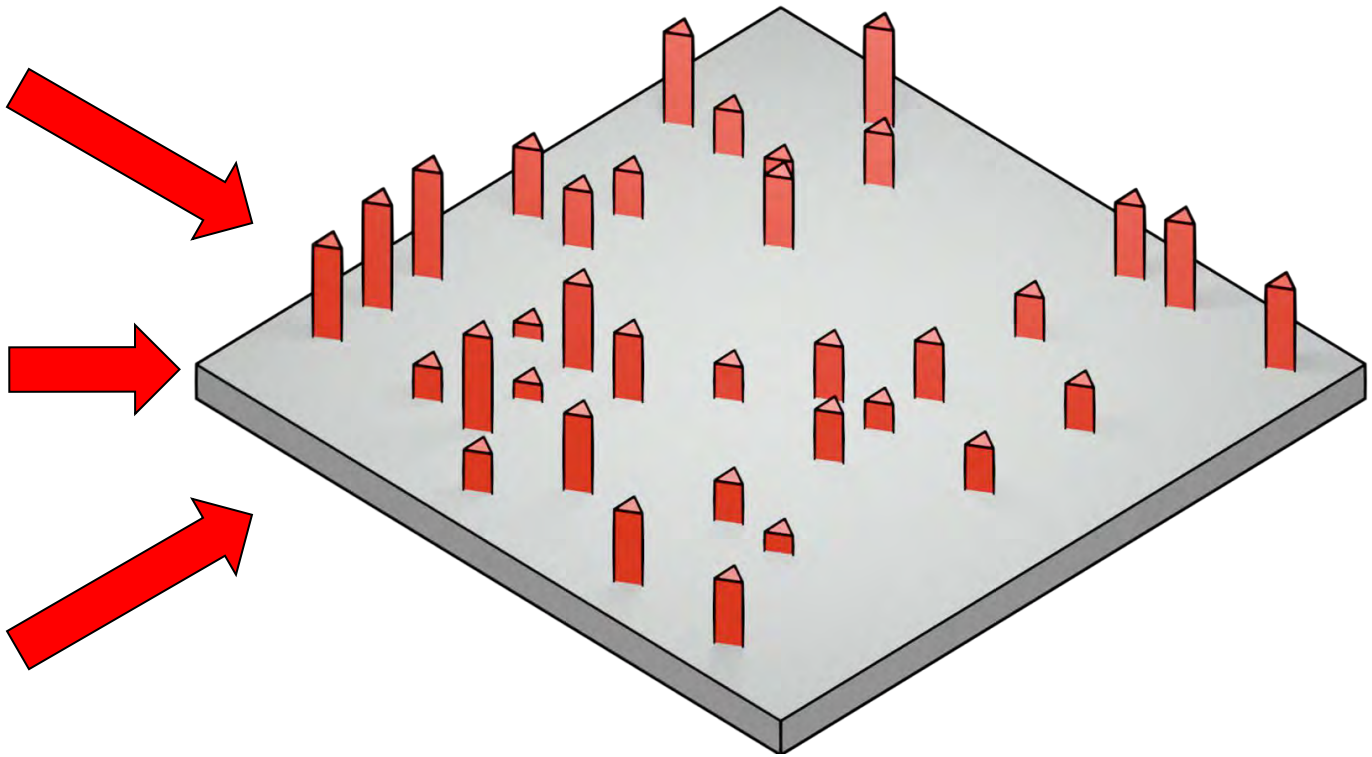
- **Feature Extraction**: Compressing 5 minutes of motion into a "Universal Action Space.“

Construction of UAS



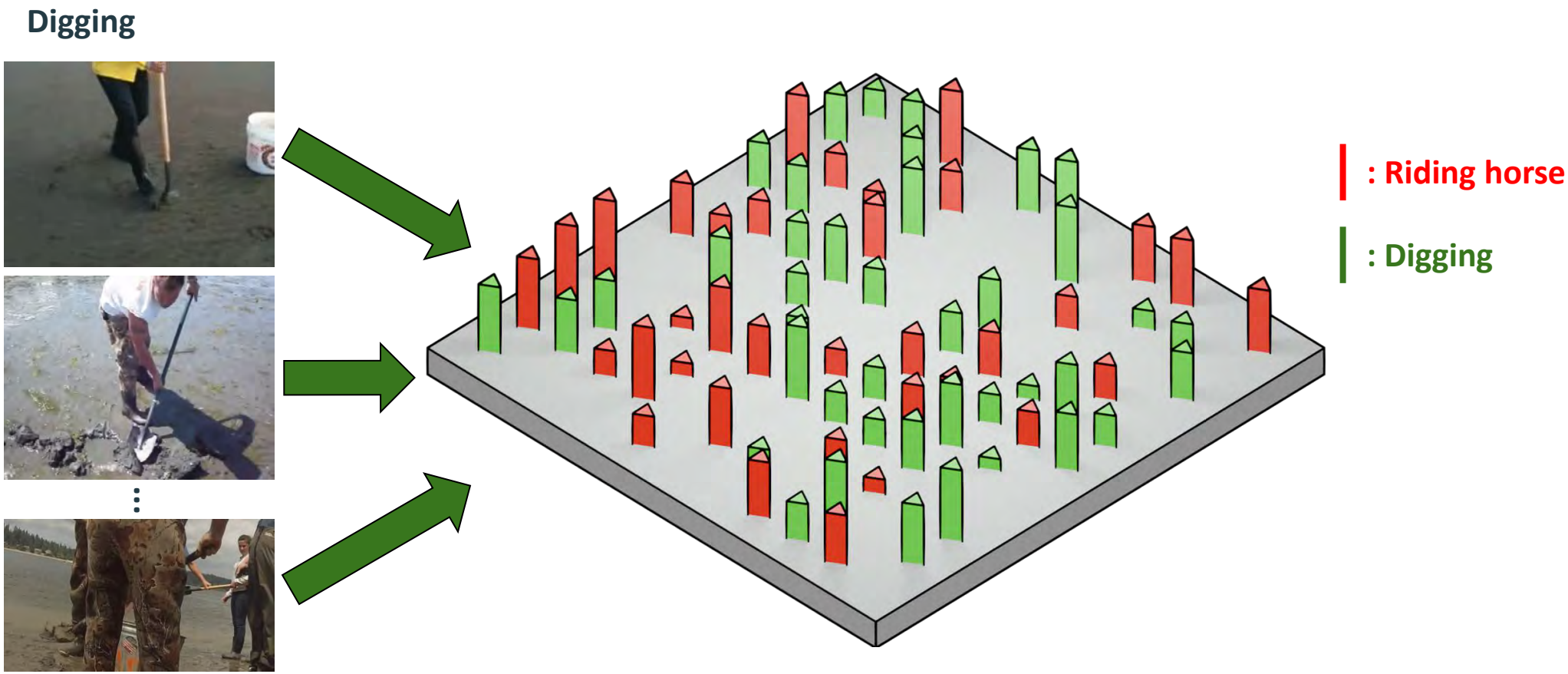
Construction of UAS

Riding horse

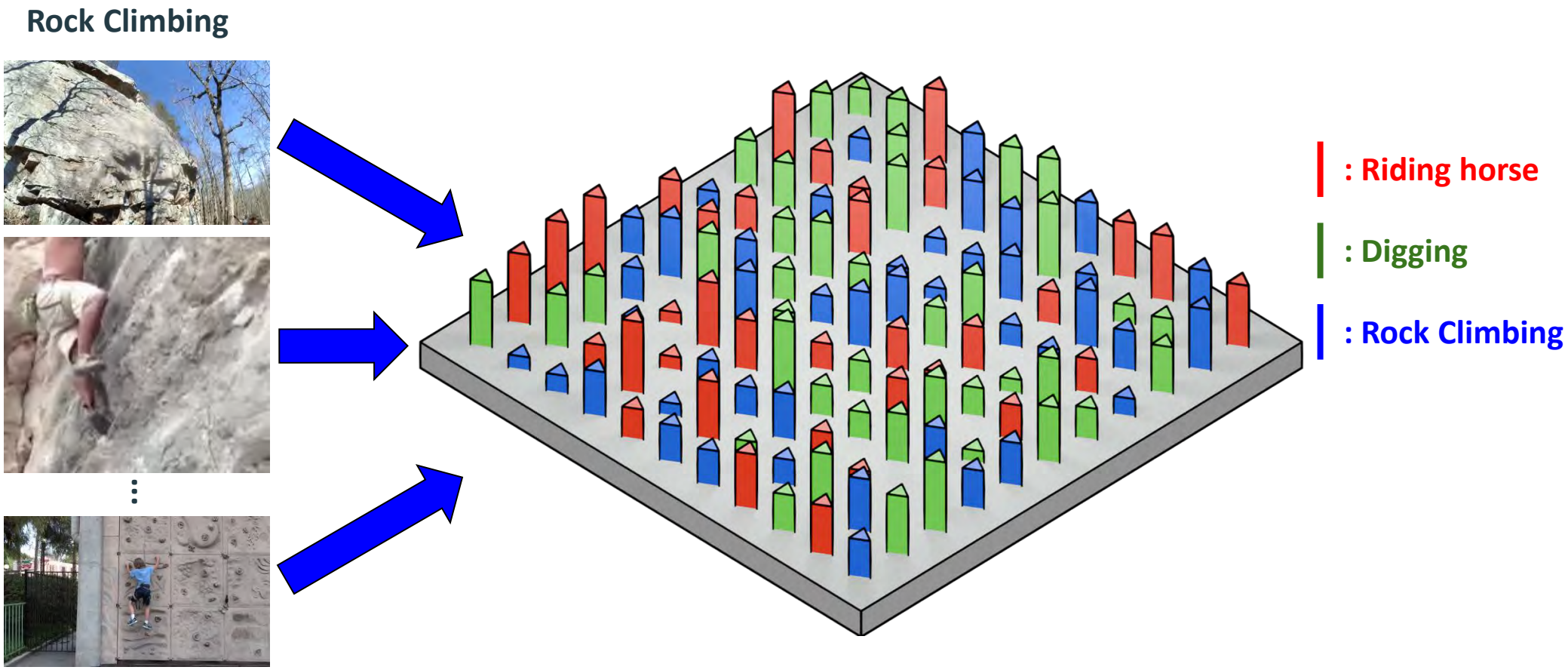


: Riding horse

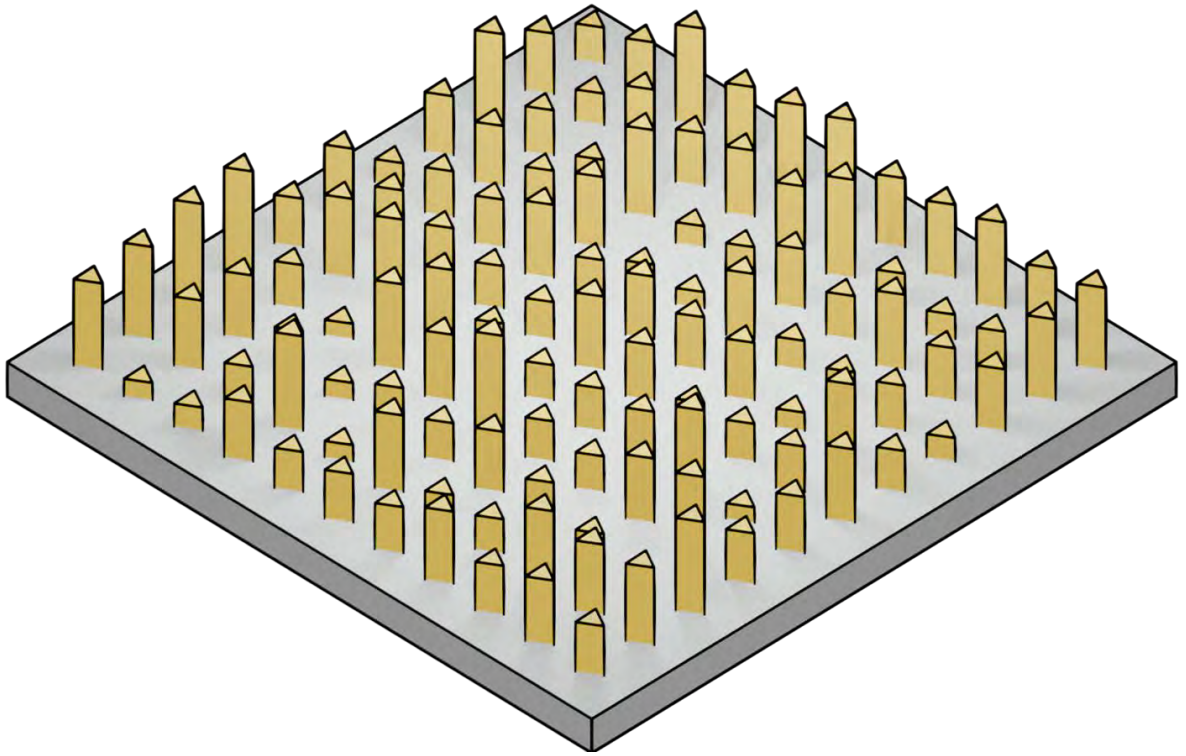
Construction of UAS



Construction of UAS



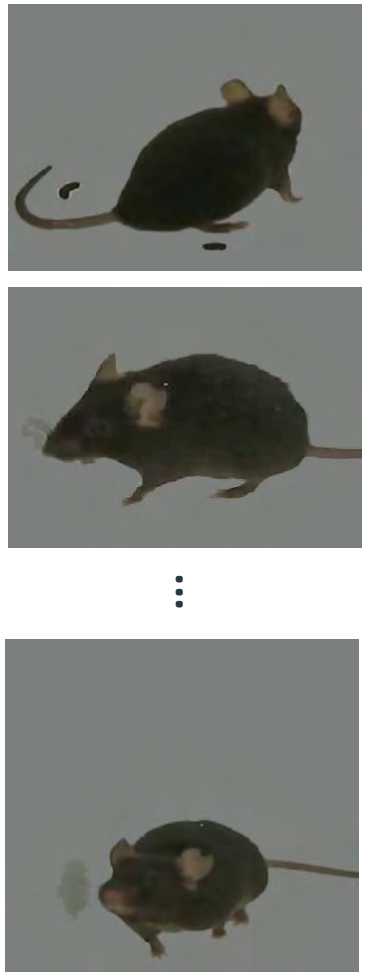
Construction of UAS

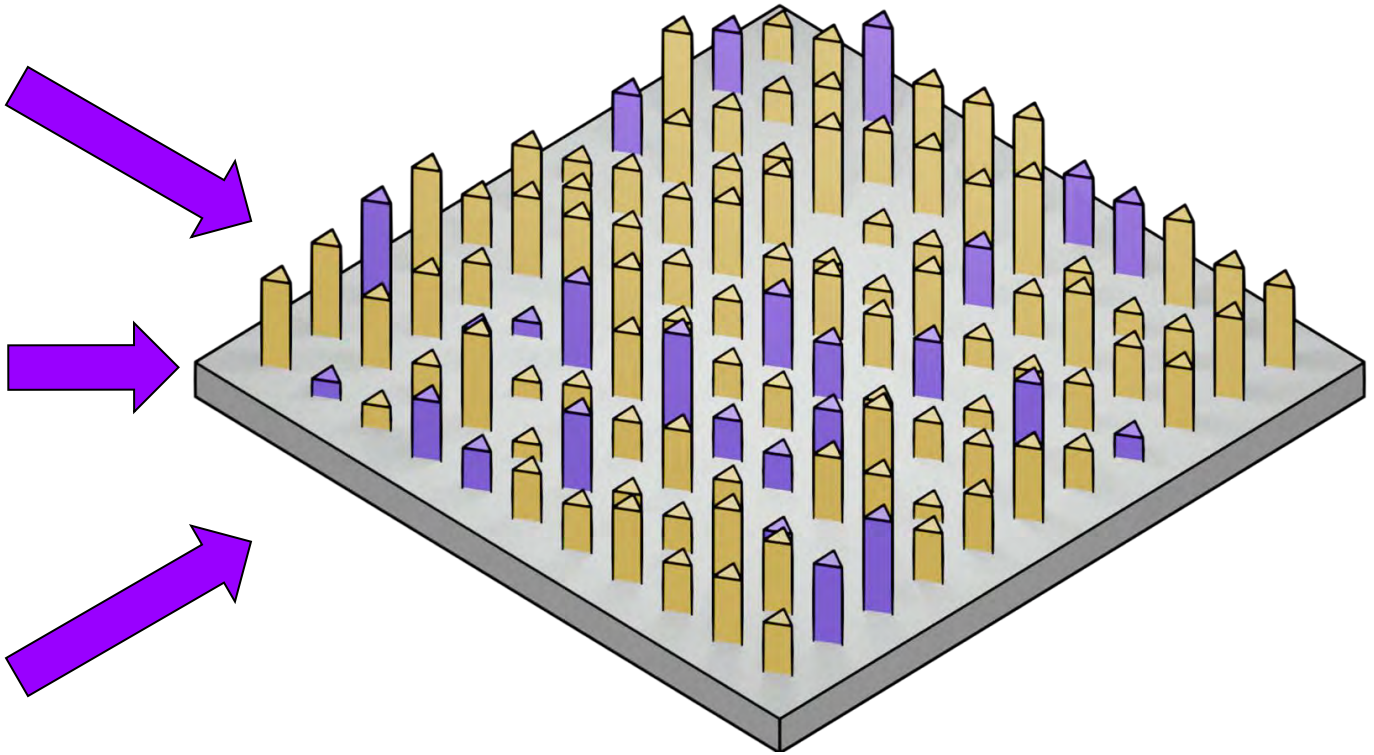


: UAS

Cross-Species Feature Mapping

Mouse behaviors

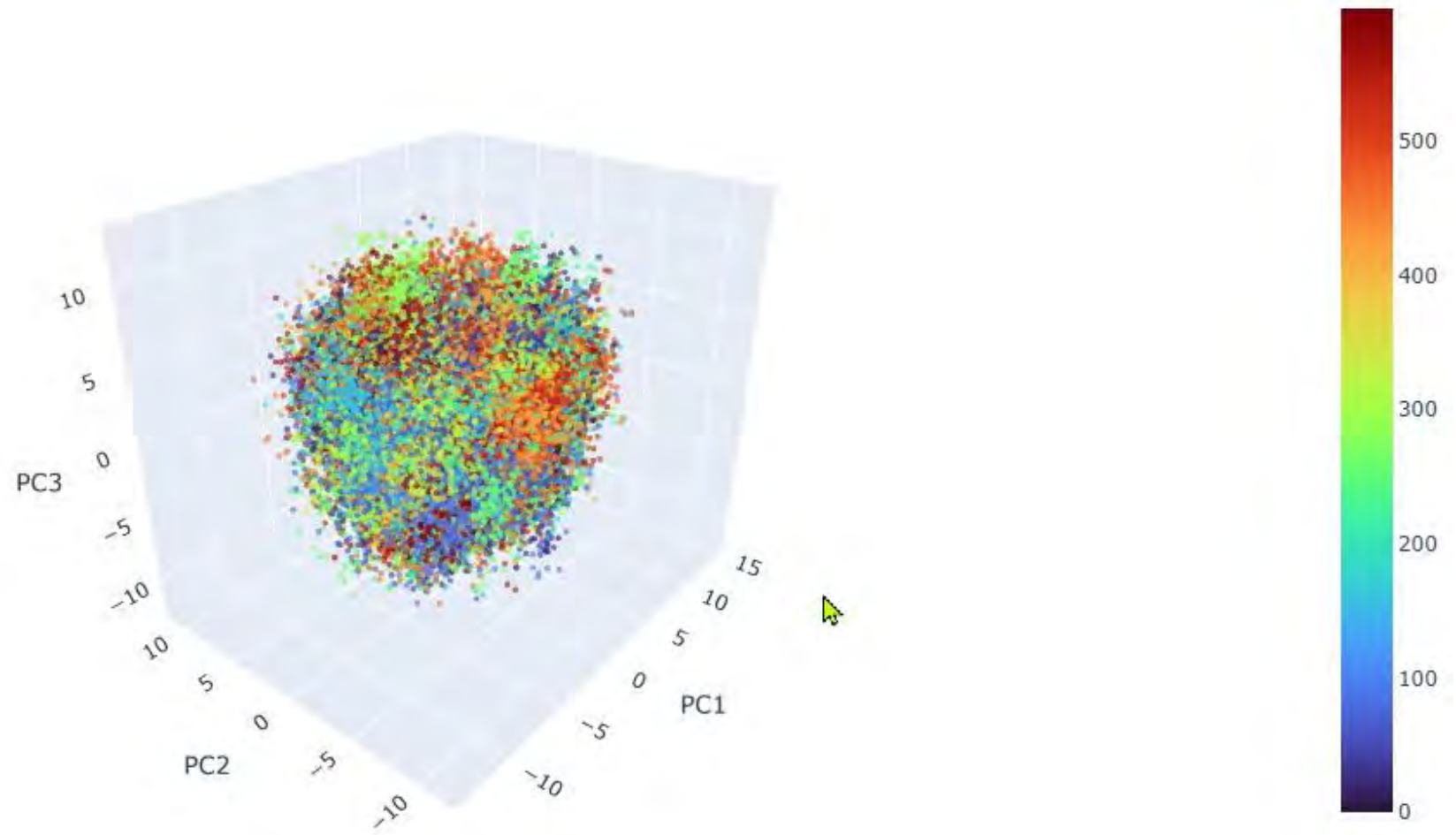




: UAS

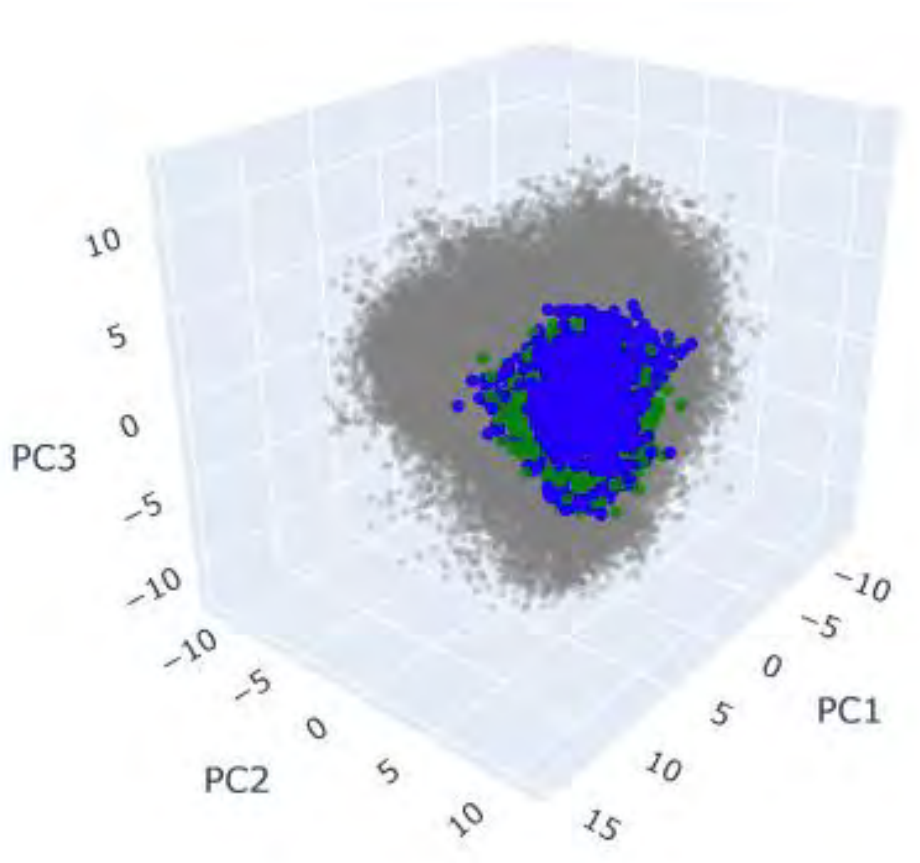
: Domain-Specific subspace

Visualizing the True UAS Embedding Space by PCA



Each point represents a 1.5s video clips from K600.
Colors indicate different action classes.

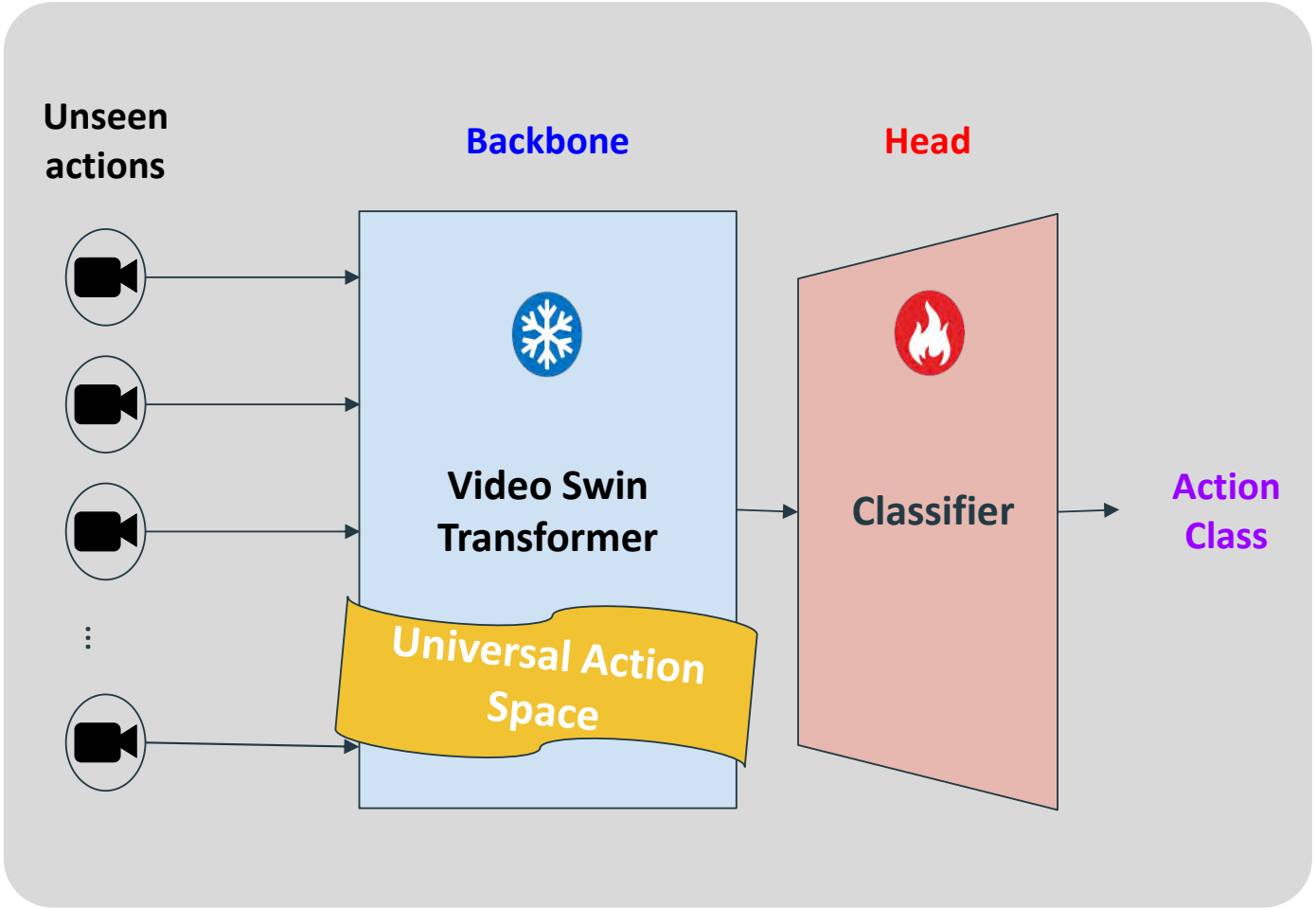
Cross-Species Alignment: Mouse Behaviors in Human UAS

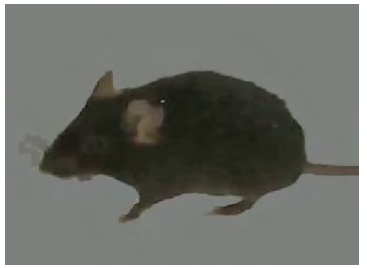


Gray: Human Base (K600 Space)
Blue: AD Mouse Data
Green: Healthy Mouse Data

Mouse behavioral patterns are fully captured within the Human UAS.

UAS-Powered Classifier





5 min video

Timeline:

1.5 seconds clips

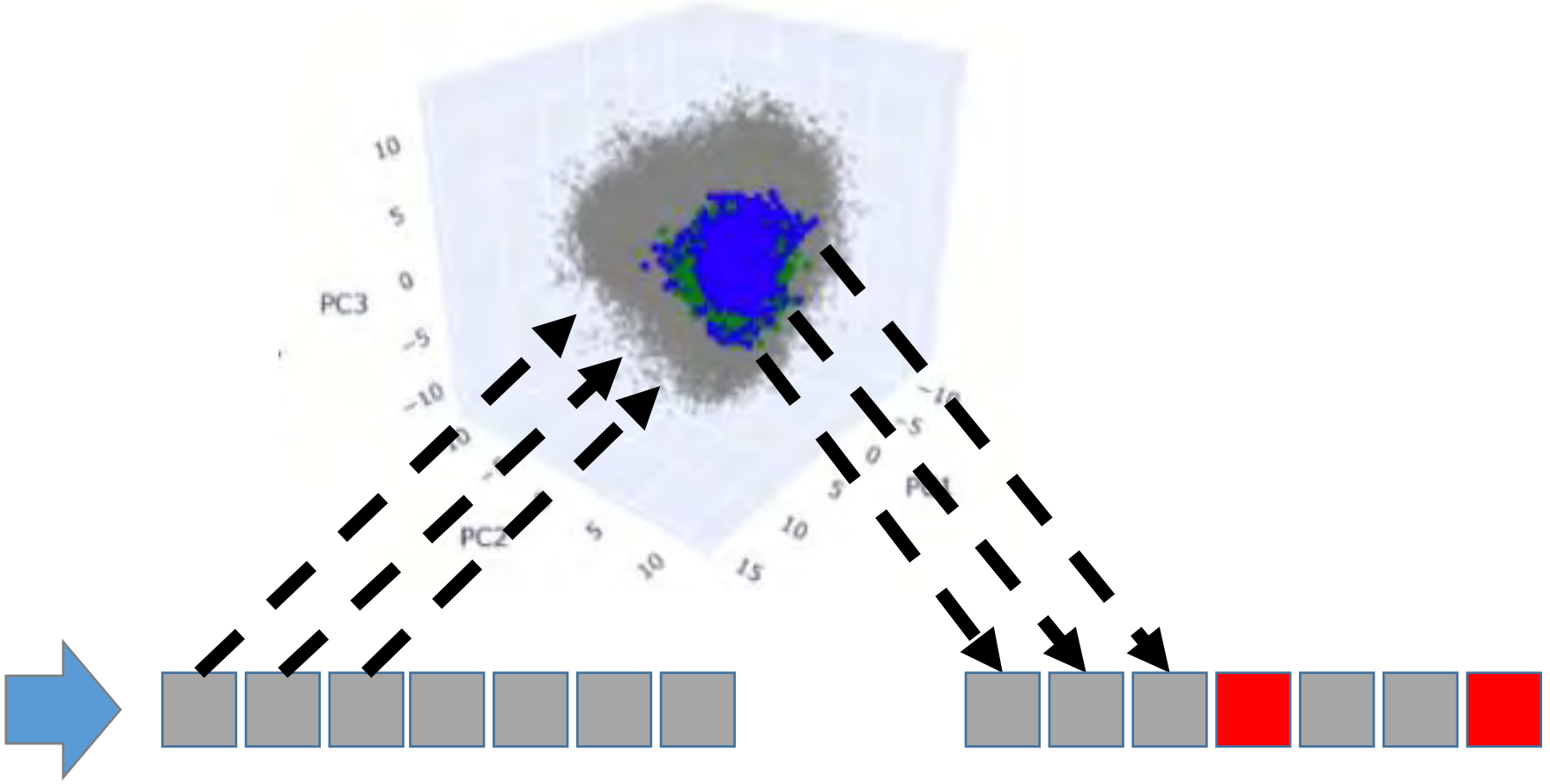
classification (video swin transformer)

Red : abnormal clips

Gray : healthy clips

UAS Feature Mapping & Classification

5 min mouse video

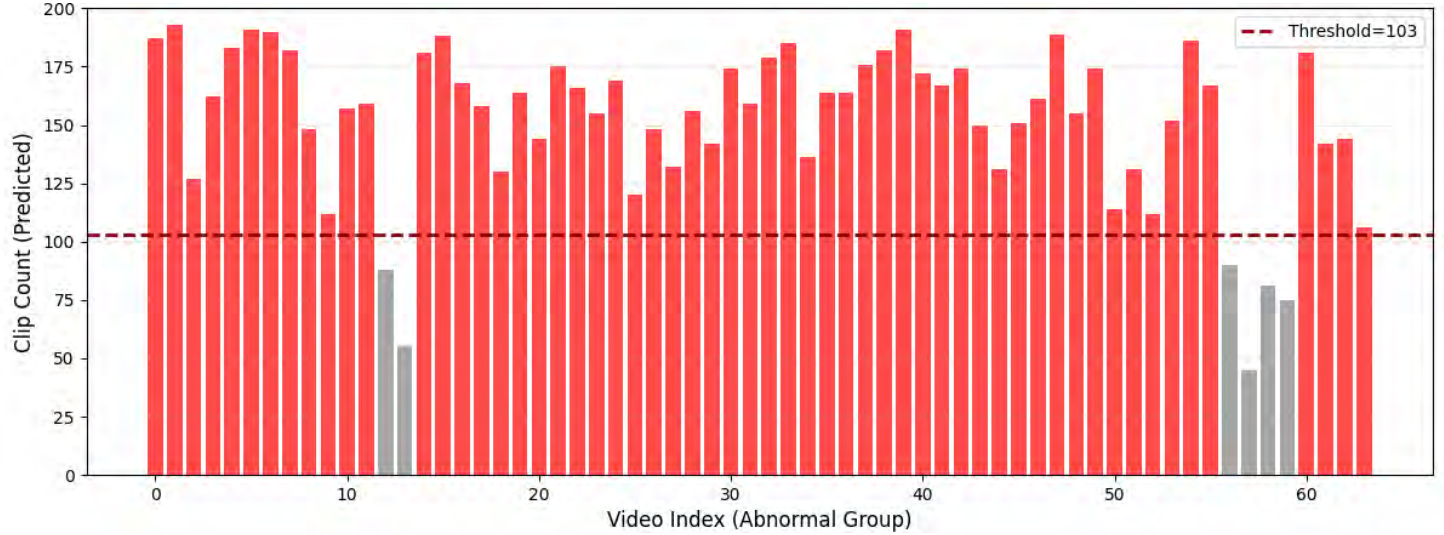


Maps clips into the UAS feature space.

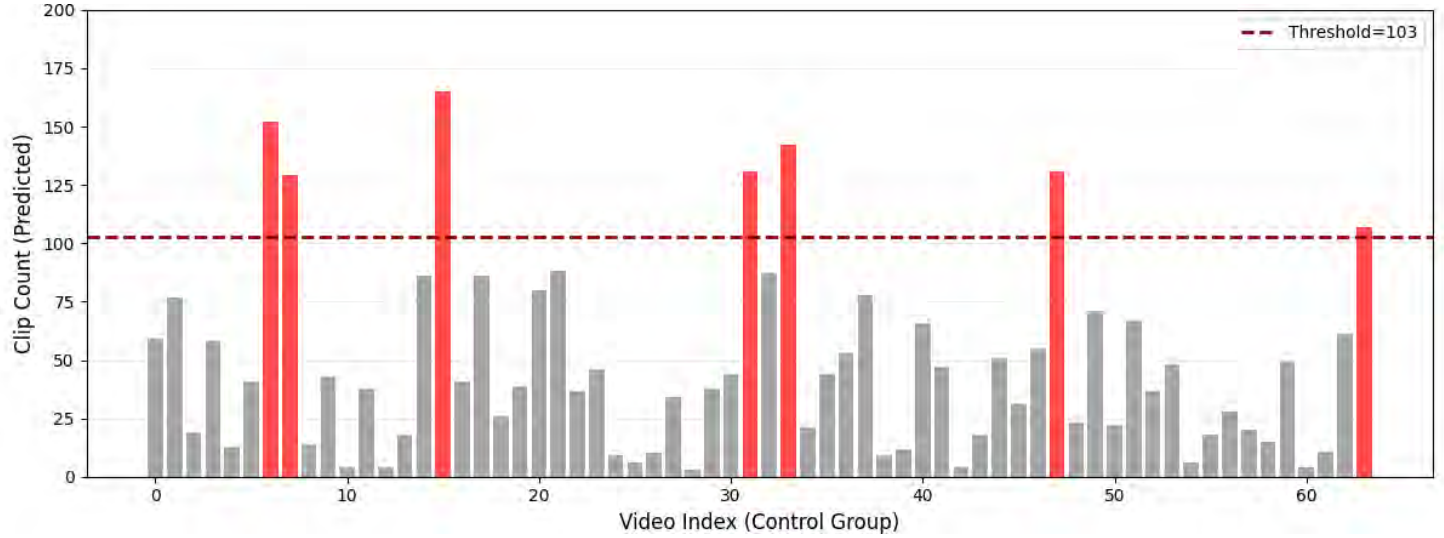
Classifies abnormal behaviors via a non-linear FC layer (latent similarity computation).

Statistical Evidence

- **X-axis**: Individual Video Index
- **Y-axis:** Detected Abnormal Clips
- **Red Line**: Optimized Threshold (k=103)



Abnormal clips in AD videos

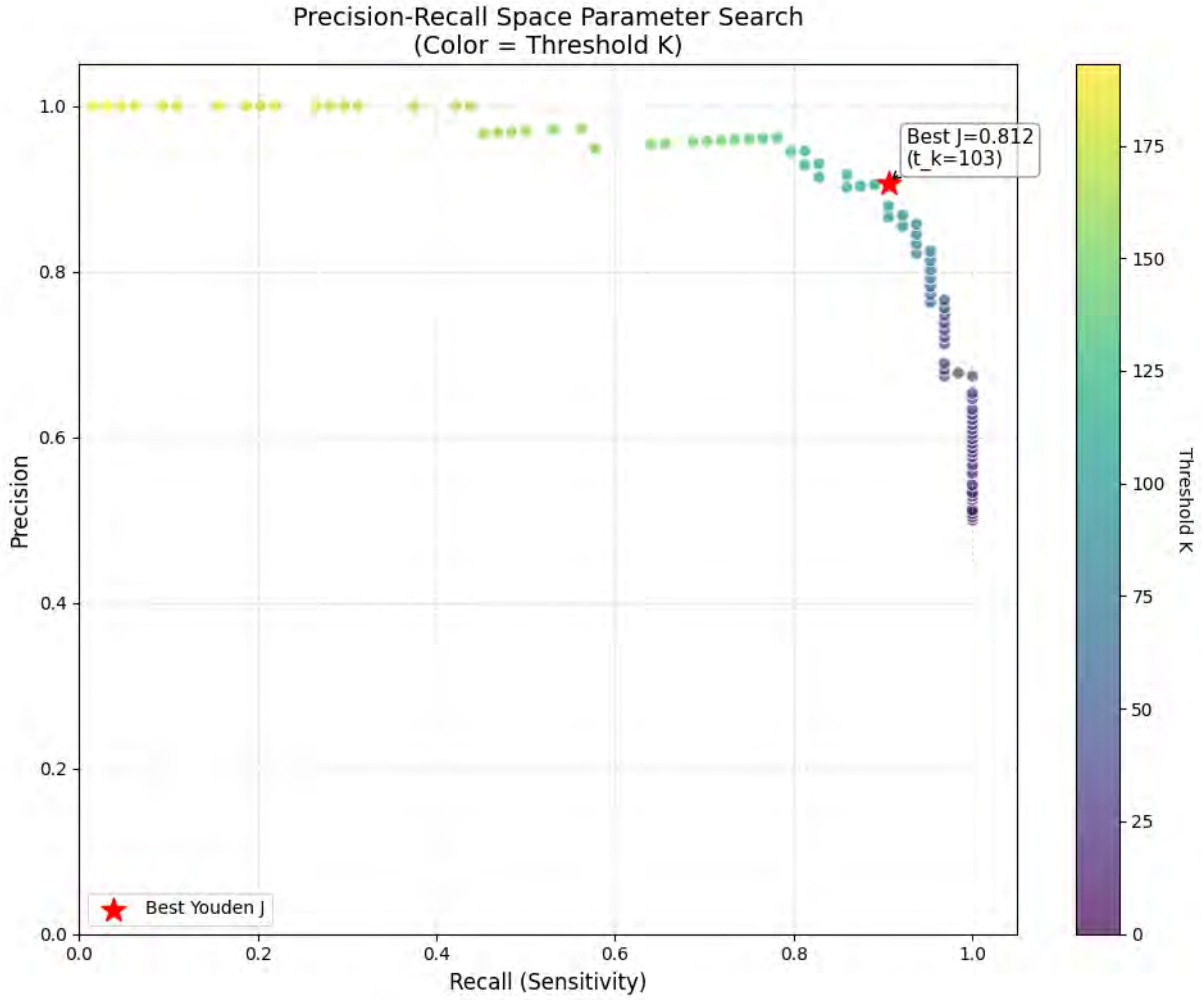


Abnormal clips in Healthy videos

Results

- Threshold optimized via Youden's J statistic with Precision-Recall curve.

	Precision	Sensitivity (Recall)	Specificity	Top-1 accuracy	Earliest detection weeks
Ours	94.38	88.30	95.08	91.80	28



YOLOv4

This is PyTorch implementation of YOLOv4 which is based on ultralytics/yolov3.

- [original Darknet implementation of YOLOv4]
- [ultralytics/yolov5 based PyTorch implementation of YOLOv4].

development log

▶ **Expand**

- `2021-10-31` - support RS loss, aLRP loss, AP loss.
- `2021-10-30` - support alpha IoU.
- `2021-10-20` - design resolution calibration methods.
- `2021-10-15` - support joint detection, instance segmentation, and semantic segmentation. `seg-yolo`
- `2021-10-13` - design ratio yolo.
- `2021-09-22` - pytorch 1.9 compatibility.
- `2021-09-21` - support DIM.
- `2021-09-16` - support Dynamic Head.
- `2021-08-28` - design domain adaptive training.
- `2021-08-22` - design re-balance models.
- `2021-08-21` - support simOTA.
- `2021-08-14` - design approximation-based methods.
- `2021-07-27` - design new decoders.
- `2021-07-22` - support 1) decoupled head, 2) anchor-free, and 3) multi positives in yolox.
- `2021-07-10` - design distribution-based implicit modeling.
- `2021-07-06` - support outlooker attention. `volo`
- `2021-07-06` - design self emsemble training method.
- `2021-06-23` - design cross multi-stage correlation module.
- `2021-06-18` - design cross stage cross correlation module.
- `2021-06-17` - support cross correlation module. `ccn`
- `2021-06-17` - support attention modules. `cbam saan`
- `2021-04-20` - support swin transformer. `swin`
- `2021-03-16` - design new stem layers.
- `2021-03-13` - design implicit modeling. `nn mf lc`
- `2021-01-26` - support vision transformer. `tr`
- `2021-01-26` - design mask objectness.
- `2021-01-25` - design rotate augmentation.
- `2021-01-23` - design collage augmentation.
- `2021-01-22` - support VoVNet, VoVNetv2.
- `2021-01-22` - support EIoU.
- `2021-01-19` - support instance segmentation. `mask-yolo`
- `2021-01-17` - support anchor-free-based methods. `center-yolo`
- `2021-01-14` - support joint detection and classification. `classify-yolo`

- 2020-01-02 - design new PRN and CSP-based models.
- 2020-12-22 - support transfer learning.
- 2020-12-18 - support non-local series self-attention blocks. gc dnl
- 2020-12-16 - support down-sampling blocks in cspnet paper. down-c down-d
- 2020-12-03 - support imitation learning.
- 2020-12-02 - support squeeze and excitation.
- 2020-11-26 - support multi-class multi-anchor joint detection and embedding.
- 2020-11-25 - support joint detection and embedding. track-yolo
- 2020-11-23 - support teacher-student learning.
- 2020-11-17 - pytorch 1.7 compatibility.
- 2020-11-06 - support inference with initial weights.
- 2020-10-21 - fully supported by darknet.
- 2020-09-18 - design fine-tune methods.
- 2020-08-29 - support deformable kernel.
- 2020-08-25 - pytorch 1.6 compatibility.
- 2020-08-24 - support channel last training/testing.
- 2020-08-16 - design CSPPRN.
- 2020-08-15 - design deeper model. csp-p6-mish
- 2020-08-11 - support HarDNet. hard39-pacsp hard68-pacsp hard85-pacsp
- 2020-08-10 - add DDP training.
- 2020-08-06 - support DCN, DCNv2. yolov4-dcn
- 2020-08-01 - add pytorch hub.
- 2020-07-31 - support ResNet, ResNeXt, CSPResNet, CSPResNeXt. r50-pacsp x50-pacsp cspr50-pacsp cspx50-pacsp
- 2020-07-28 - support SAM. yolov4-pacsp-sam
- 2020-07-24 - update api.
- 2020-07-23 - support CUDA accelerated Mish activation function.
- 2020-07-19 - support and training tiny YOLOv4. yolov4-tiny
- 2020-07-15 - design and training conditional YOLOv4. yolov4-pacsp-conditional
- 2020-07-13 - support MixUp data augmentation.
- 2020-07-03 - design new stem layers.
- 2020-06-16 - support floating16 of GPU inference.
- 2020-06-14 - convert .pt to .weights for darknet fine-tuning.
- 2020-06-13 - update multi-scale training strategy.
- 2020-06-12 - design scaled YOLOv4 follow ultralytics. yolov4-pacsp-s yolov4-pacsp-m yolov4-pacsp-l yolov4-pacsp-x
- 2020-06-07 - design scaling methods for CSP-based models. yolov4-pacsp-25 yolov4-pacsp-75
- 2020-06-03 - update COCO2014 to COCO2017.
- 2020-05-30 - update FPN neck to CSPFPN. yolov4-yocsp yolov4-yocsp-mish
- 2020-05-24 - update neck of YOLOv4 to CSPPAN. yolov4-pacsp yolov4-pacsp-mish
- 2020-05-15 - training YOLOv4 with Mish activation function. yolov4-yospp-mish yolov4-paspp-mish
- 2020-05-08 - design and training YOLOv4 with FPN neck. yolov4-yospp
- 2020-05-01 - training YOLOv4 with Leaky activation function using PyTorch. yolov4-paspp PAN

Pretrained Models & Comparison

Model	Test Size	AP^{test}	AP_{50}^{test}	AP_{75}^{test}	AP_S^{test}	AP_M^{test}	AP_L^{test}	cfg	weights
YOLOv4	640	50.0%	68.4%	54.7%	30.5%	54.3%	63.3%	cfg	weights
YOLOv4$_{pacsp-s}$	640	39.0%	57.8%	42.4%	20.6%	42.6%	50.0%	cfg	weights
YOLOv4$_{pacsp}$	640	49.8%	68.4%	54.3%	30.1%	54.0%	63.4%	cfg	weights
YOLOv4$_{pacsp-x}$	640	**52.2%**	**70.5%**	**56.8%**	**32.7%**	**56.3%**	**65.9%**	cfg	weights
YOLOv4$_{pacsp-s-mish}$	640	40.8%	59.5%	44.3%	22.4%	44.6%	51.8%	cfg	weights
YOLOv4$_{pacsp-mish}$	640	50.9%	69.4%	55.5%	31.2%	55.0%	64.7%	cfg	weights
YOLOv4$_{pacsp-x-mish}$	640	52.8%	71.1%	57.5%	33.6%	56.9%	66.6%	cfg	weights

Model	Test Size	AP^{val}	AP_{50}^{val}	AP_{75}^{val}	AP_S^{val}	AP_M^{val}	AP_L^{val}	cfg	weights
YOLOv4	640	49.7%	68.2%	54.3%	32.9%	54.8%	63.7%	cfg	weights
YOLOv4$_{pacsp-s}$	640	38.9%	57.7%	42.2%	21.9%	43.3%	51.9%	cfg	weights
YOLOv4$_{pacsp}$	640	49.4%	68.1%	53.8%	32.7%	54.2%	64.0%	cfg	weights
YOLOv4$_{pacsp-x}$	640	**51.6%**	**70.1%**	**56.2%**	**35.3%**	**56.4%**	**66.9%**	cfg	weights
YOLOv4$_{pacsp-s-mish}$	640	40.7%	59.5%	44.2%	25.3%	45.1%	53.4%	cfg	weights
YOLOv4$_{pacsp-mish}$	640	50.8%	69.4%	55.4%	34.3%	55.5%	65.7%	cfg	weights
YOLOv4$_{pacsp-x-mish}$	640	52.6%	71.0%	57.2%	36.4%	57.3%	67.6%	cfg	weights

▶ **archive**

Model	Test Size	AP^{val}	AP_{50}^{val}	AP_{75}^{val}	AP_S^{val}	AP_M^{val}	AP_L^{val}	cfg	weights
YOLOv4	640	48.4%	67.1%	52.9%	31.7%	53.8%	62.0%	cfg	weights

Model	Test Size	AP^{val}	AP_{50}^{val}	AP_{75}^{val}	AP_S^{val}	AP_M^{val}	AP_L^{val}	cfg	weights
YOLOv4$_{pacsp-s}$	640	37.0%	55.7%	40.0%	20.2%	41.6%	48.4%	cfg	weights
YOLOv4$_{pacsp}$	640	47.7%	66.4%	52.0%	32.3%	53.0%	61.7%	cfg	weights
YOLOv4$_{pacsp-x}$	640	**50.0%**	**68.3%**	**54.5%**	**33.9%**	**55.4%**	**63.7%**	cfg	weights
YOLOv4$_{pacsp-s-mish}$	640	38.8%	57.8%	42.0%	21.6%	43.7%	51.1%	cfg	weights
YOLOv4$_{pacsp-mish}$	640	48.8%	67.2%	53.4%	31.5%	54.4%	62.2%	cfg	weights
YOLOv4$_{pacsp-x-mish}$	640	51.2%	69.4%	55.9%	35.0%	56.5%	65.0%	cfg	weights
Model	**Test Size**	**AP^{val}**	**AP_{50}^{val}**	**AP_{75}^{val}**	**AP_S^{val}**	**AP_M^{val}**	**AP_L^{val}**	**cfg**	**weights**
YOLOv4	672	47.7%	66.7%	52.1%	30.5%	52.6%	61.4%	cfg	weights
YOLOv4$_{pacsp-s}$	672	36.6%	55.5%	39.6%	21.2%	41.1%	47.0%	cfg	weights
YOLOv4$_{pacsp}$	672	47.2%	66.2%	51.6%	30.4%	52.3%	60.8%	cfg	weights
YOLOv4$_{pacsp-x}$	672	**49.3%**	**68.1%**	**53.6%**	**31.8%**	**54.5%**	**63.6%**	cfg	weights
YOLOv4$_{pacsp-s-mish}$	672	38.6%	57.7%	41.8%	22.3%	43.5%	49.3%	cfg	weights
(+BoF)	640	39.9%	59.1%	43.1%	24.4%	45.2%	51.4%		weights
YOLOv4$_{pacsp-mish}$	672	48.1%	66.9%	52.3%	30.8%	53.4%	61.7%	cfg	weights
(+BoF)	640	49.3%	68.2%	53.8%	31.9%	54.9%	62.8%		weights
YOLOv4$_{pacsp-x-mish}$	672	50.0%	68.5%	54.4%	32.9%	54.9%	64.0%	cfg	weights
(+BoF)	640	**51.0%**	**69.7%**	**55.5%**	**33.3%**	**56.2%**	**65.5%**		weights

Requirements

docker (recommanded):

```
# create the docker container, you can change the share memory size if you
have more.
nvidia-docker run --name yolov4 -it -v your_coco_path/:/coco/ -v
your_code_path/:/yolo --shm-size=64g nvcr.io/nvidia/pytorch:20.11-py3

# apt install required packages
apt update
apt install -y zip htop screen libgl1-mesa-glx

# pip install required packages
pip install seaborn thop

# install mish-cuda if you want to use mish activation
# https://github.com/thomasbrandon/mish-cuda
# https://github.com/JunnYu/mish-cuda
cd /
git clone https://github.com/JunnYu/mish-cuda
cd mish-cuda
python setup.py build install

# go to code folder
cd /yolo
```

local:

```
pip install -r requirements.txt
```

※ For running Mish models, please install https://github.com/thomasbrandon/mish-cuda

Training

```
python train.py --device 0 --batch-size 16 --img 640 640 --data coco.yaml -
-cfg cfg/yolov4-pacsp.cfg --weights '' --name yolov4-pacsp
```

Testing

```
python test.py --img 640 --conf 0.001 --batch 8 --device 0 --data coco.yaml
--cfg cfg/yolov4-pacsp.cfg --weights weights/yolov4-pacsp.pt
```

Citation

```
@article{bochkovskiy2020yolov4,
  title={{YOLOv4}: Optimal Speed and Accuracy of Object Detection},
```

```
  author={Bochkovskiy, Alexey and Wang, Chien-Yao and Liao, Hong-Yuan
Mark},
  journal={arXiv preprint arXiv:2004.10934},
  year={2020}
}
```

```
@inproceedings{wang2020cspnet,
  title={{CSPNet}: A New Backbone That Can Enhance Learning Capability of
{CNN}},
  author={Wang, Chien-Yao and Mark Liao, Hong-Yuan and Wu, Yueh-Hua and
Chen, Ping-Yang and Hsieh, Jun-Wei and Yeh, I-Hau},
  booktitle={Proceedings of the IEEE/CVF Conference on Computer Vision and
Pattern Recognition Workshops},
  pages={390--391},
  year={2020}
}
```

Acknowledgements

- https://github.com/AlexeyAB/darknet
- https://github.com/ultralytics/yolov3
- https://github.com/ultralytics/yolov5

Official YOLOv7

Implementation of paper - YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors

custom badge unparseable json response | Hugging Face Spaces | Open in Colab | cs.CV arXiv:2207.02696

Web Demo

- Integrated into Huggingface Spaces 🤗 using Gradio. Try out the Web Demo Hugging Face Spaces

Performance

MS COCO

Model	Test Size	AP^{test}	AP_{50}^{test}	AP_{75}^{test}	batch 1 fps	batch 32 average time
YOLOv7	640	**51.4%**	**69.7%**	**55.9%**	161 *fps*	2.8 *ms*
YOLOv7-X	640	**53.1%**	**71.2%**	**57.8%**	114 *fps*	4.3 *ms*
YOLOv7-W6	1280	**54.9%**	**72.6%**	**60.1%**	84 *fps*	7.6 *ms*
YOLOv7-E6	1280	**56.0%**	**73.5%**	**61.2%**	56 *fps*	12.3 *ms*
YOLOv7-D6	1280	**56.6%**	**74.0%**	**61.8%**	44 *fps*	15.0 *ms*
YOLOv7-E6E	1280	**56.8%**	**74.4%**	**62.1%**	36 *fps*	18.7 *ms*

Installation

Docker environment (recommended)

▶ **Expand**

```
# create the docker container, you can change the share memory size if you
have more.
nvidia-docker run --name yolov7 -it -v your_coco_path/:/coco/ -v
your_code_path/:/yolov7 --shm-size=64g nvcr.io/nvidia/pytorch:21.08-py3

# apt install required packages
apt update
apt install -y zip htop screen libgl1-mesa-glx

# pip install required packages
pip install seaborn thop
```

```
# go to code folder
cd /yolov7
```

Testing

`yolov7.pt` `yolov7x.pt` `yolov7-w6.pt` `yolov7-e6.pt` `yolov7-d6.pt` `yolov7-e6e.pt`

```
python test.py --data data/coco.yaml --img 640 --batch 32 --conf 0.001 --
iou 0.65 --device 0 --weights yolov7.pt --name yolov7_640_val
```

You will get the results:

```
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] =
0.51206
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] =
0.69730
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] =
0.55521
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] =
0.35247
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] =
0.55937
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] =
0.66693
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] =
0.38453
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] =
0.63765
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] =
0.68772
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] =
0.53766
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] =
0.73549
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] =
0.83868
```

To measure accuracy, download COCO-annotations for Pycocotools to the `./coco/annotations/instances_val2017.json`

Training

Data preparation

```
bash scripts/get_coco.sh
```

- Download MS COCO dataset images (train, val, test) and labels. If you have previously used a different version of YOLO, we strongly recommend that you delete `train2017.cache` and `val2017.cache` files, and redownload labels

Single GPU training

```
# train p5 models
python train.py --workers 8 --device 0 --batch-size 32 --data
data/coco.yaml --img 640 640 --cfg cfg/training/yolov7.yaml --weights '' --
name yolov7 --hyp data/hyp.scratch.p5.yaml

# train p6 models
python train_aux.py --workers 8 --device 0 --batch-size 16 --data
data/coco.yaml --img 1280 1280 --cfg cfg/training/yolov7-w6.yaml --weights
'' --name yolov7-w6 --hyp data/hyp.scratch.p6.yaml
```

Multiple GPU training

```
# train p5 models
python -m torch.distributed.launch --nproc_per_node 4 --master_port 9527
train.py --workers 8 --device 0,1,2,3 --sync-bn --batch-size 128 --data
data/coco.yaml --img 640 640 --cfg cfg/training/yolov7.yaml --weights '' --
name yolov7 --hyp data/hyp.scratch.p5.yaml

# train p6 models
python -m torch.distributed.launch --nproc_per_node 8 --master_port 9527
train_aux.py --workers 8 --device 0,1,2,3,4,5,6,7 --sync-bn --batch-size
128 --data data/coco.yaml --img 1280 1280 --cfg cfg/training/yolov7-w6.yaml
--weights '' --name yolov7-w6 --hyp data/hyp.scratch.p6.yaml
```

Transfer learning

`yolov7_training.pt` `yolov7x_training.pt` `yolov7-w6_training.pt` `yolov7-e6_training.pt` `yolov7-d6_training.pt` `yolov7-e6e_training.pt`

Single GPU finetuning for custom dataset

```
# finetune p5 models
python train.py --workers 8 --device 0 --batch-size 32 --data
data/custom.yaml --img 640 640 --cfg cfg/training/yolov7-custom.yaml --
weights 'yolov7_training.pt' --name yolov7-custom --hyp
data/hyp.scratch.custom.yaml

# finetune p6 models
python train_aux.py --workers 8 --device 0 --batch-size 16 --data
data/custom.yaml --img 1280 1280 --cfg cfg/training/yolov7-w6-custom.yaml -
-weights 'yolov7-w6_training.pt' --name yolov7-w6-custom --hyp
data/hyp.scratch.custom.yaml
```

Re-parameterization

See reparameterization.ipynb

Inference

On video:

```
python detect.py --weights yolov7.pt --conf 0.25 --img-size 640 --source
yourvideo.mp4
```

On image:

```
python detect.py --weights yolov7.pt --conf 0.25 --img-size 640 --source
inference/images/horses.jpg
```

Export

Pytorch to CoreML (and inference on MacOS/iOS) Open in Colab

Pytorch to ONNX with NMS (and inference) Open in Colab

```
python export.py --weights yolov7-tiny.pt --grid --end2end --simplify \
        --topk-all 100 --iou-thres 0.65 --conf-thres 0.35 --img-size 640
640 --max-wh 640
```

Pytorch to TensorRT with NMS (and inference) Open in Colab

```
wget https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-
tiny.pt
python export.py --weights ./yolov7-tiny.pt --grid --end2end --simplify --
topk-all 100 --iou-thres 0.65 --conf-thres 0.35 --img-size 640 640
git clone https://github.com/Linaom1214/tensorrt-python.git
python ./tensorrt-python/export.py -o yolov7-tiny.onnx -e yolov7-tiny-
nms.trt -p fp16
```

Pytorch to TensorRT another way Open in Colab

▶ **Expand**

YOLOv9

Implementation of paper - YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information

cs.CV arXiv:2402.13616 Hugging Face Spaces Hugging Face Spaces Open in Colab OpenCV BlogPost

Performance

MS COCO

Model	Test Size	AP^{val}	AP_{50}^{val}	AP_{75}^{val}	Param.	FLOPs
YOLOv9-T	640	**38.3%**	**53.1%**	**41.3%**	**2.0M**	**7.7G**
YOLOv9-S	640	**46.8%**	**63.4%**	**50.7%**	**7.1M**	**26.4G**
YOLOv9-M	640	**51.4%**	**68.1%**	**56.1%**	**20.0M**	**76.3G**
YOLOv9-C	640	**53.0%**	**70.2%**	**57.8%**	**25.3M**	**102.1G**
YOLOv9-E	640	**55.6%**	**72.8%**	**60.6%**	**57.3M**	**189.0G**
<!--	**YOLOv9 (ReLU)**	640	**51.9%**	**69.1%**	**56.5%**	**25.3M**

Useful Links

▶ **Expand**

Custom training: https://github.com/WongKinYiu/yolov9/issues/30#issuecomment-1960955297

ONNX export: https://github.com/WongKinYiu/yolov9/issues/2#issuecomment-1960519506
https://github.com/WongKinYiu/yolov9/issues/40#issue-2150697688 https://github.com/WongKinYiu/yolov9/issues/130#issue-2162045461

ONNX export for segmentation: https://github.com/WongKinYiu/yolov9/issues/260#issue-2191162150

TensorRT inference: https://github.com/WongKinYiu/yolov9/issues/143#issuecomment-1975049660
https://github.com/WongKinYiu/yolov9/issues/34#issue-2150393690 https://github.com/WongKinYiu/yolov9/issues/79#issue-2153547004
https://github.com/WongKinYiu/yolov9/issues/143#issue-2164002309

QAT TensorRT: https://github.com/WongKinYiu/yolov9/issues/327#issue-2229284136 https://github.com/WongKinYiu/yolov9/issues/253#issue-2189520073

TensorRT inference for segmentation: https://github.com/WongKinYiu/yolov9/issues/446

TFLite: https://github.com/WongKinYiu/yolov9/issues/374#issuecomment-2065751706

OpenVINO: https://github.com/WongKinYiu/yolov9/issues/164#issue-2168540003

C# ONNX inference: https://github.com/WongKinYiu/yolov9/issues/95#issue-2155974619

C# OpenVINO inference: https://github.com/WongKinYiu/yolov9/issues/95#issuecomment-1968131244

OpenCV: https://github.com/WongKinYiu/yolov9/issues/113#issuecomment-1971327672

Hugging Face demo: https://github.com/WongKinYiu/yolov9/issues/45#issuecomment-1961496943

CoLab demo: https://github.com/WongKinYiu/yolov9/pull/18

ONNXSlim export: https://github.com/WongKinYiu/yolov9/pull/37

YOLOv9 ROS: https://github.com/WongKinYiu/yolov9/issues/144#issue-2164210644

YOLOv9 ROS TensorRT: https://github.com/WongKinYiu/yolov9/issues/145#issue-2164218595

YOLOv9 Julia: https://github.com/WongKinYiu/yolov9/issues/141#issuecomment-1973710107

YOLOv9 MLX: https://github.com/WongKinYiu/yolov9/issues/258#issue-2190586540

YOLOv9 StrongSORT with OSNet: https://github.com/WongKinYiu/yolov9/issues/299#issue-2212093340

YOLOv9 ByteTrack: https://github.com/WongKinYiu/yolov9/issues/78#issue-2153512879

YOLOv9 DeepSORT: https://github.com/WongKinYiu/yolov9/issues/98#issue-2156172319

YOLOv9 counting: https://github.com/WongKinYiu/yolov9/issues/84#issue-2153904804

YOLOv9 speed estimation: https://github.com/WongKinYiu/yolov9/issues/456

YOLOv9 face detection: https://github.com/WongKinYiu/yolov9/issues/121#issue-2160218766

YOLOv9 segmentation onnxruntime: https://github.com/WongKinYiu/yolov9/issues/151#issue-2165667350

Comet logging: https://github.com/WongKinYiu/yolov9/pull/110

MLflow logging: https://github.com/WongKinYiu/yolov9/pull/87

AnyLabeling tool: https://github.com/WongKinYiu/yolov9/issues/48#issue-2152139662

AX650N deploy: https://github.com/WongKinYiu/yolov9/issues/96#issue-2156115760

Conda environment: https://github.com/WongKinYiu/yolov9/pull/93

AutoDL docker environment: https://github.com/WongKinYiu/yolov9/issues/112#issue-2158203480

Installation

Docker environment (recommended)

▶ **Expand**

```
# create the docker container, you can change the share memory size if you have more.
nvidia-docker run --name yolov9 -it -v your_coco_path/:/coco/ -v your_code_path/:/yolov9 --shm-size=64g
nvcr.io/nvidia/pytorch:21.11-py3

# apt install required packages
apt update
apt install -y zip htop screen libgl1-mesa-glx

# pip install required packages
pip install seaborn thop

# go to code folder
cd /yolov9
```

Evaluation

`yolov9-s-converted.pt` `yolov9-m-converted.pt` `yolov9-c-converted.pt` `yolov9-e-converted.pt` `yolov9-s.pt` `yolov9-m.pt` `yolov9-c.pt` `yolov9-e.pt` `gelan-s.pt` `gelan-m.pt` `gelan-c.pt` `gelan-e.pt`

```
# evaluate converted yolov9 models
python val.py --data data/coco.yaml --img 640 --batch 32 --conf 0.001 --iou 0.7 --device 0 --weights
'./yolov9-c-converted.pt' --save-json --name yolov9_c_c_640_val

# evaluate yolov9 models
# python val_dual.py --data data/coco.yaml --img 640 --batch 32 --conf 0.001 --iou 0.7 --device 0 --
weights './yolov9-c.pt' --save-json --name yolov9_c_640_val

# evaluate gelan models
# python val.py --data data/coco.yaml --img 640 --batch 32 --conf 0.001 --iou 0.7 --device 0 --weights
'./gelan-c.pt' --save-json --name gelan_c_640_val
```

You will get the results:

```
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.530
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.702
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.578
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.362
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.585
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.693
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.392
```

```
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.652
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.702
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.541
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.760
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.844
```

Training

Data preparation

```
bash scripts/get_coco.sh
```

- Download MS COCO dataset images (train, val, test) and labels. If you have previously used a different version of YOLO, we strongly recommend that you delete `train2017.cache` and `val2017.cache` files, and redownload labels

Single GPU training

```
# train yolov9 models
python train_dual.py --workers 8 --device 0 --batch 16 --data data/coco.yaml --img 640 --cfg
models/detect/yolov9-c.yaml --weights '' --name yolov9-c --hyp hyp.scratch-high.yaml --min-items 0 --
epochs 500 --close-mosaic 15

# train gelan models
# python train.py --workers 8 --device 0 --batch 32 --data data/coco.yaml --img 640 --cfg
models/detect/gelan-c.yaml --weights '' --name gelan-c --hyp hyp.scratch-high.yaml --min-items 0 --epochs
500 --close-mosaic 15
```

Multiple GPU training

```
# train yolov9 models
python -m torch.distributed.launch --nproc_per_node 8 --master_port 9527 train_dual.py --workers 8 --
device 0,1,2,3,4,5,6,7 --sync-bn --batch 128 --data data/coco.yaml --img 640 --cfg models/detect/yolov9-
c.yaml --weights '' --name yolov9-c --hyp hyp.scratch-high.yaml --min-items 0 --epochs 500 --close-mosaic
15

# train gelan models
# python -m torch.distributed.launch --nproc_per_node 4 --master_port 9527 train.py --workers 8 --device
0,1,2,3 --sync-bn --batch 128 --data data/coco.yaml --img 640 --cfg models/detect/gelan-c.yaml --weights
'' --name gelan-c --hyp hyp.scratch-high.yaml --min-items 0 --epochs 500 --close-mosaic 15
```

Re-parameterization

See reparameterization.ipynb.

Inference

```
# inference converted yolov9 models
python detect.py --source './data/images/horses.jpg' --img 640 --device 0 --weights './yolov9-c-
converted.pt' --name yolov9_c_c_640_detect

# inference yolov9 models
# python detect_dual.py --source './data/images/horses.jpg' --img 640 --device 0 --weights './yolov9-c.pt'
--name yolov9_c_640_detect

# inference gelan models
# python detect.py --source './data/images/horses.jpg' --img 640 --device 0 --weights './gelan-c.pt' --
name gelan_c_c_640_detect
```

Citation

```
@article{wang2024yolov9,
  title={{YOLOv9}: Learning What You Want to Learn Using Programmable Gradient Information},
  author={Wang, Chien-Yao  and Liao, Hong-Yuan Mark},
  booktitle={arXiv preprint arXiv:2402.13616},
  year={2024}
}
```

```
@article{chang2023yolor,
  title={{YOLOR}-Based Multi-Task Learning},
  author={Chang, Hung-Shuo and Wang, Chien-Yao and Wang, Richard Robert and Chou, Gene and Liao, Hong-Yuan
Mark},
  journal={arXiv preprint arXiv:2309.16921},
  year={2023}
}
```

Teaser

Parts of code of YOLOR-Based Multi-Task Learning are released in the repository.

Object Detection

`gelan-c-det.pt`

`object detection`

```
# coco/labels/{split}/*.txt
# bbox or polygon (1 instance 1 line)
python train.py --workers 8 --device 0 --batch 32 --data data/coco.yaml --img 640 --cfg
models/detect/gelan-c.yaml --weights '' --name gelan-c-det --hyp hyp.scratch-high.yaml --min-items 0 --
epochs 300 --close-mosaic 10
```

Model	Test Size	Param.	FLOPs	AP^{box}
GELAN-C-DET	640	25.3M	102.1G	**52.3%**
YOLOv9-C-DET	640	25.3M	102.1G	**53.0%**

Instance Segmentation

`gelan-c-seg.pt`

`object detection` `instance segmentation`

```
# coco/labels/{split}/*.txt
# polygon (1 instance 1 line)
python segment/train.py --workers 8 --device 0 --batch 32  --data coco.yaml --img 640 --cfg
models/segment/gelan-c-seg.yaml --weights '' --name gelan-c-seg --hyp hyp.scratch-high.yaml --no-overlap -
-epochs 300 --close-mosaic 10
```

Model	Test Size	Param.	FLOPs	AP^{box}	AP^{mask}
GELAN-C-SEG	640	27.4M	144.6G	**52.3%**	**42.4%**
YOLOv9-C-SEG	640	27.4M	145.5G	**53.3%**	**43.5%**

Panoptic Segmentation

`gelan-c-pan.pt`

`object detection` `instance segmentation` `semantic segmentation` `stuff segmentation` `panoptic segmentation`

```
# coco/labels/{split}/*.txt
# polygon (1 instance 1 line)
# coco/stuff/{split}/*.txt
# polygon (1 semantic 1 line)
python panoptic/train.py --workers 8 --device 0 --batch 32  --data coco.yaml --img 640 --cfg
models/panoptic/gelan-c-pan.yaml --weights '' --name gelan-c-pan --hyp hyp.scratch-high.yaml --no-overlap
--epochs 300 --close-mosaic 10
```

Model	Test Size	Param.	FLOPs	AP^{box}	AP^{mask}	$mIoU_{164k/10k}^{semantic}$	$mIoU^{stuff}$	$PQ^{panoptic}$
GELAN-C-PAN	640	27.6M	146.7G	**52.6%**	**42.5%**	**39.0%/48.3%**	**52.7%**	**39.4%**
YOLOv9-C-PAN	640	28.8M	187.0G	**52.7%**	**43.0%**	**39.8%/-**	**52.2%**	**40.5%**

Image Captioning (not yet released)

`object detection` `instance segmentation` `semantic segmentation` `stuff segmentation` `panoptic segmentation` `image captioning`

```
# coco/labels/{split}/*.txt
# polygon (1 instance 1 line)
# coco/stuff/{split}/*.txt
# polygon (1 semantic 1 line)
# coco/annotations/*.json
# json (1 split 1 file)
python caption/train.py --workers 8 --device 0 --batch 32  --data coco.yaml --img 640 --cfg
models/caption/gelan-c-cap.yaml --weights '' --name gelan-c-cap --hyp hyp.scratch-high.yaml --no-overlap -
-epochs 300 --close-mosaic 10
```

Model	Test Size	Param.	FLOPs	AP^{box}	AP^{mask}	$mIoU_{164k/10k}^{semantic}$	$mIoU^{stuff}$	$PQ^{panoptic}$	$BLEU@4^{caption}$	$CIDEr^{caption}$
GELAN-C-CAP	640	47.5M	-	**51.9%**	**42.6%**	**42.5%/-**	**56.5%**	**41.7%**	**38.8**	**122.3**
YOLOv9-C-CAP	640	47.5M	-	**52.1%**	**42.6%**	**43.0%/-**	**56.4%**	**42.1%**	**39.1**	**122.0**
<!--	**YOLOR-MT**	640	79.3M	-	**51.0%**	**41.7%**	**-/49.6%**	**55.9%**	**40.5%**	**35.7**

Acknowledgements

▶ **Expand**

- https://github.com/AlexeyAB/darknet
- https://github.com/WongKinYiu/yolor
- https://github.com/WongKinYiu/yolov7
- https://github.com/VDIGPKU/DynamicDet
- https://github.com/DingXiaoH/RepVGG
- https://github.com/ultralytics/yolov5
- https://github.com/meituan/YOLOv6

GeneralistYOLO

Generalist YOLO: Towards Real-Time End-to-End Multi-Task Visual Language Models

Performance

Generalist Visual Language Tasks

MS COCO

Model	Size	#Param.	FLOPs	$AP_{e2e/nms}^{box}$	$AP_{e2e/nms}^{mask}$	$mIoU_{164k/10k}^{semantic}$	$mIoU^{stuff}$	$PQ^{panoptic}$	$BLEU@4^{caption}$	$CIDEr^{ca}$
GeneralistYOLO	640	32.5M	~122.2G	**52.4%/52.8%**	**43.0%/43.1%**	**44.7%/52.0%**	**59.2%**	**44.2%**	**38.7**	**122.**
GYOLO	640	32.5M	~122.2G	**52.3%/**	**42.9%/**	**44.5%/**	**59.2%**	**44.3%**	**38.9**	**122.**
GYOLO-X	640	-	-	**53.3%/**	**43.8%/**	**46.0%/**	**59.9%**	**45.4%**	**39.2**	**123.**
<!--	****		M	G	**%/%**	**%/%**	**%/%**	**%**	**%**	****

Installation

Generalist Visual Language Tasks

Docker environment (recommended)

▶ **expand**

```
# create the docker container, you can change the share memory size if you have more.
nvidia-docker run --name gyolo -it -v your_coco_path/:/coco/ -v your_code_path/:/gyolo --shm-size=64g
nvcr.io/nvidia/pytorch:21.11-py3
```

Data preparation

▶ **expand**

Packages

```
pip install -r requirements.txt
```

Note: java is required for pycocoevalcap tool

Dataset

We use MS COCO Dataset.
Please download the following files:

- Images
  - 2017 Train images
  - 2017 Val images
- Annotations
  - 2017 Train/Val annotations
  - 2017 Stuff Train/Val annotations
  - 2017 Panoptic Train/Val annotations

Labels

Download the labels from:

- COCO
- Segmentation labels Extract these zip files and put the data of `Segmentation labels` into `coco/stuff` folder.

Modify Yaml File

Modify path of `data/coco.yaml` to use the dataset:

```
path: path_to_coco_dir
train: train2017.txt
val: val2017.txt
```

Dataset Folder Hierarchy

The folder hierarchy should be:

```
coco/
├─ annotations/
│   ├─ captions_train2017.json
│   ├─ captions_val2017.json
│   ├─ instances_train2017.json
│   ├─ instances_val2017.json
│   ├─ panoptic_train2017.json
│   ├─ panoptic_val2017.json
│   ├─ stuff_train2017.json
│   └─ stuff_val2017.json
├─ images/
│   ├─ train2017/
│   │   └─ *.jpg
│   └─ val2017/
│       └─ *.jpg
├─ labels/
│   ├─ train2017/
│   │   └─ *.txt
│   └─ val2017/
│       └─ *.txt
├─ panoptic/
│   └─ masks/
│       ├─ train2017/
│       │   └─ *.png
│       └─ val2017/
│           └─ *.png
├─ stuff/
│   ├─ train2017/
│   │   └─ *.txt
│   └─ val2017/
│       └─ *.txt
├─ train2017.txt
└─ val2017.txt
```

Evaluation

Generalist Visual Language Tasks

```
python caption/val.py --data coco.yaml --batch 4 --weights 'gyolo.pt' --img 640 --conf 0.001 --iou 0.7 --device 0 --
save-json --export-mask
```

▶ **End-to-End Performance**

Object Detection

```
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.524
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.700
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.569
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.343
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.580
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.680
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.388
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.646
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.695
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.529
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.754
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.843
```

Instance Segmentation

```
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.430
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.669
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.457
```

```
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.232
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.480
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.611
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.337
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.538
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.571
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.365
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.633
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.766
```

Sementic Segmentation (164k)

Class	IoU	Acc
unlabeled	nan	nan
person	84.86	93.16
bicycle	70.03	85.5
car	64.03	80.68
motorcycle	82.11	92.03
airplane	83.58	95.03
bus	83.89	88.31
train	85.93	95.77
truck	58.29	77.87
boat	60.76	80.73
traffic light	72.42	88.09
fire hydrant	87.81	96.83
stop sign	88.85	98.47
parking meter	71.55	79.2
bench	48.87	72.44
bird	70.25	80.34
cat	85.84	94.42
dog	77.41	86.99
horse	82.46	96.0
sheep	86.14	95.76
cow	84.38	91.59
elephant	90.21	97.53
bear	89.74	94.42
zebra	88.84	95.87
giraffe	86.82	95.68
backpack	29.63	53.67
umbrella	74.51	87.73
handbag	26.91	46.98
tie	1.52	7.45
suitcase	72.03	88.68
frisbee	33.48	88.33
skis	48.53	63.79
snowboard	54.1	62.81
sports ball	61.87	74.01
kite	54.46	71.27
baseball bat	44.28	63.32
baseball glove	45.31	75.14
skateboard	59.39	69.72
surfboard	69.36	82.31
tennis racket	77.2	94.05
bottle	60.69	76.84
wine glass	71.92	83.25
cup	56.67	78.08
fork	50.22	59.83
knife	36.22	45.91
spoon	38.49	48.59
bowl	52.75	64.88
banana	74.19	90.02
apple	46.75	66.49
sandwich	52.52	73.81
orange	62.68	90.65
broccoli	47.92	69.18
carrot	55.95	72.26
hot dog	62.78	71.53
pizza	80.19	90.89
donut	62.88	81.4
cake	63.41	80.72
chair	48.88	70.26
couch	52.75	81.58
potted plant	32.8	52.98
bed	62.34	80.18
dining table	46.45	77.74
toilet	80.01	93.89
tv	70.21	85.8

laptop	76.01	92.74
mouse	71.09	90.58
remote	41.18	74.18
keyboard	76.8	86.66
cell phone	69.47	88.76
microwave	71.89	80.98
oven	57.07	85.7
toaster	16.02	19.88
sink	54.85	81.12
refrigerator	72.63	91.97
book	43.71	67.28
clock	66.02	81.57
vase	57.18	81.76
scissors	57.41	68.47
teddy bear	75.06	94.48
hair drier	12.1	13.0
toothbrush	42.57	82.4
banner	28.75	64.1
blanket	4.71	4.9
branch	17.54	19.07
bridge	29.11	41.19
building-other	53.07	71.46
bush	28.35	35.92
cabinet	53.94	73.32
cage	22.65	37.39
cardboard	38.95	52.3
carpet	48.63	70.33
ceiling-other	59.97	83.41
ceiling-tile	0.0	0.0
cloth	0.0	0.0
clothes	15.54	19.18
clouds	51.44	70.78
counter	27.17	49.03
cupboard	0.0	0.0
curtain	61.09	75.68
desk-stuff	41.19	58.88
dirt	39.18	63.3
door-stuff	34.88	58.0
fence	32.11	55.61
floor-marble	0.0	0.0
floor-other	20.38	28.56
floor-stone	1.22	1.45
floor-tile	59.88	73.49
floor-wood	56.31	76.05
flower	41.24	63.94
fog	5.29	5.53
food-other	23.03	29.74
fruit	23.25	33.46
furniture-other	11.95	14.54
grass	67.63	84.0
gravel	24.22	33.02
ground-other	0.01	0.01
hill	11.37	14.06
house	24.15	28.92
leaves	19.67	23.16
light	33.13	43.04
mat	0.0	0.0
metal	25.95	35.11
mirror-stuff	46.3	59.93
moss	0.0	0.0
mountain	51.84	69.33
mud	0.0	0.0
napkin	0.0	0.0
net	38.73	56.65
paper	26.97	37.97
pavement	47.96	65.6
pillow	0.04	0.04
plant-other	11.92	17.66
plastic	8.67	10.36
platform	26.13	43.2
playingfield	67.44	85.82
railing	0.0	0.0
railroad	58.88	80.49
river	47.47	73.29
road	63.05	85.34
rock	49.4	78.22
roof	13.73	16.23
rug	31.78	47.12
salad	0.0	0.0
sand	59.8	66.58
sea	84.48	93.09

```
|      shelf       | 28.05 | 44.77 |
|    sky-other     | 71.52 | 85.74 |
|    skyscraper    | 34.83 | 45.86 |
|       snow       | 87.85 | 94.18 |
|   solid-other    |  0.0  |  0.0  |
|      stairs      | 18.45 | 34.88 |
|      stone       |  0.04 |  0.04 |
|      straw       | 25.23 | 35.38 |
| structural-other |  0.0  |  0.0  |
|      table       | 21.97 | 32.77 |
|       tent       |  7.99 | 10.02 |
|  textile-other   | 11.05 | 16.53 |
|      towel       | 26.33 | 34.97 |
|       tree       | 71.69 | 89.39 |
|    vegetable     | 29.58 | 39.57 |
|    wall-brick    | 44.48 |  65.1 |
|  wall-concrete   | 56.56 | 77.85 |
|    wall-other    | 17.42 | 26.48 |
|    wall-panel    |  0.0  |  0.0  |
|    wall-stone    | 28.92 |  35.3 |
|    wall-tile     | 62.57 |  81.9 |
|    wall-wood     | 36.82 | 52.74 |
|   water-other    | 24.21 | 32.21 |
|    waterdrops    |  0.0  |  0.0  |
|   window-blind   | 43.78 | 55.65 |
|   window-other   | 41.27 | 68.13 |
|       wood       | 14.98 | 20.71 |
|      other       |  nan  |  nan  |
+------------------+-------+-------+
mIoU: 44.71
```

Sementic Segmentation (10k)

```
+------------------+-------+-------+
|      Class       |  IoU  |  Acc  |
+------------------+-------+-------+
|    unlabeled     |  0.0  |  0.0  |
|      person      | 89.51 | 93.95 |
|     bicycle      | 80.23 | 94.89 |
|       car        | 77.79 | 91.13 |
|    motorcycle    | 91.53 | 96.09 |
|     airplane     | 91.56 | 96.86 |
|       bus        | 78.32 | 82.81 |
|      train       | 84.03 | 97.35 |
|      truck       | 81.86 | 91.39 |
|       boat       | 82.03 |  91.1 |
|  traffic light   | 83.19 | 92.57 |
|   fire hydrant   | 94.32 | 97.83 |
|    stop sign     |  96.3 |  98.8 |
|  parking meter   | 93.45 | 98.75 |
|      bench       |  63.4 | 73.69 |
|       bird       | 78.83 | 83.16 |
|       cat        | 92.11 |  96.2 |
|       dog        | 90.22 | 96.16 |
|      horse       | 91.82 | 96.67 |
|      sheep       | 84.12 | 88.55 |
|       cow        | 95.46 |  98.8 |
|     elephant     | 91.32 | 97.54 |
|       bear       | 94.91 | 98.01 |
|      zebra       | 91.93 | 96.91 |
|     giraffe      | 88.71 | 95.79 |
|     backpack     | 26.41 | 64.16 |
|     umbrella     | 74.86 | 82.87 |
|     handbag      | 19.46 | 28.59 |
|       tie        | 56.53 | 87.15 |
|     suitcase     |  78.7 | 96.52 |
|     frisbee      | 94.25 | 98.44 |
|       skis       | 53.84 | 71.81 |
|    snowboard     |  78.0 | 89.26 |
|   sports ball    | 80.14 | 92.07 |
|       kite       | 82.64 | 90.41 |
|   baseball bat   | 61.12 |  88.1 |
|  baseball glove  | 87.25 | 96.25 |
|    skateboard    | 71.44 | 92.04 |
|    surfboard     | 92.07 | 95.83 |
|  tennis racket   | 86.32 | 98.22 |
|      bottle      | 62.63 | 67.93 |
|    wine glass    | 83.16 | 94.68 |
|       cup        | 74.92 | 84.94 |
```

```
|       fork       | 42.35 | 61.53 |
|      knife       | 75.21 | 79.47 |
|      spoon       | 53.88 | 59.72 |
|       bowl       | 65.96 |  70.1 |
|      banana      | 84.71 | 94.33 |
|      apple       | 73.57 | 84.38 |
|     sandwich     | 87.26 | 88.74 |
|      orange      | 68.71 | 91.81 |
|     broccoli     | 87.19 | 94.84 |
|      carrot      | 73.76 | 84.37 |
|     hot dog      | 48.05 | 96.32 |
|      pizza       | 93.58 | 95.03 |
|      donut       | 62.73 | 73.17 |
|       cake       | 91.87 | 93.91 |
|      chair       |  66.2 | 87.91 |
|      couch       | 77.24 | 95.24 |
|   potted plant   | 45.71 | 56.18 |
|       bed        | 81.64 | 92.91 |
|   dining table   | 66.72 | 88.02 |
|      toilet      | 92.97 | 97.25 |
|        tv        | 78.62 | 96.39 |
|      laptop      | 88.94 | 97.86 |
|      mouse       | 75.67 | 79.79 |
|      remote      | 51.58 | 62.81 |
|     keyboard     | 90.94 | 95.07 |
|    cell phone    | 84.11 | 97.22 |
|    microwave     | 41.51 | 76.53 |
|       oven       | 73.16 | 88.12 |
|     toaster      | 19.62 | 19.62 |
|       sink       | 84.92 | 89.03 |
|   refrigerator   | 87.58 | 97.52 |
|       book       | 77.65 | 90.83 |
|      clock       | 74.15 | 92.76 |
|       vase       | 68.39 | 96.87 |
|     scissors     | 82.61 | 96.54 |
|    teddy bear    | 87.25 | 98.11 |
|    hair drier    |  0.94 |  0.94 |
|    toothbrush    | 25.65 |  30.9 |
|      banner      | 40.85 |  63.5 |
|     blanket      |  0.0  |  0.0  |
|      branch      |  0.0  |  0.0  |
|      bridge      |  8.37 | 10.71 |
|  building-other  | 61.54 | 80.41 |
|       bush       | 27.83 | 30.96 |
|     cabinet      | 21.48 | 79.41 |
|       cage       |  0.0  |  0.0  |
|    cardboard     | 13.07 | 23.45 |
|      carpet      | 64.81 | 86.65 |
|  ceiling-other   | 64.85 | 86.37 |
|   ceiling-tile   |  0.0  |  0.0  |
|      cloth       |  0.0  |  0.0  |
|     clothes      | 33.63 | 45.14 |
|      clouds      | 40.58 | 45.55 |
|     counter      | 52.43 | 59.55 |
|     cupboard     |  0.0  |  0.0  |
|     curtain      | 66.88 |  81.6 |
|    desk-stuff    | 37.35 | 49.24 |
|       dirt       | 36.32 | 70.93 |
|    door-stuff    | 49.48 | 68.39 |
|      fence       | 41.34 | 57.21 |
|   floor-marble   |  0.0  |  0.0  |
|   floor-other    | 35.03 | 58.93 |
|   floor-stone    | 14.11 | 21.97 |
|    floor-tile    | 61.11 | 73.52 |
|    floor-wood    | 66.94 | 85.74 |
|      flower      | 20.39 | 54.75 |
|       fog        |  0.0  |  0.0  |
|    food-other    | 30.71 | 43.36 |
|      fruit       |  65.3 | 84.93 |
| furniture-other  | 17.89 | 20.81 |
|      grass       | 75.38 | 89.23 |
|      gravel      | 29.83 | 35.46 |
|   ground-other   |  0.0  |  0.0  |
|       hill       | 26.77 | 29.34 |
|      house       | 41.49 | 52.24 |
|      leaves      | 46.16 | 59.14 |
|      light       | 34.34 |  43.4 |
|       mat        |  0.0  |  0.0  |
|      metal       | 12.79 | 30.98 |
|   mirror-stuff   | 53.17 | 62.22 |
|       moss       |  0.0  |  0.0  |
```

```
|     mountain     | 36.63 | 91.39 |
|       mud        |  0.0  |  0.0  |
|      napkin      |  0.0  |  0.0  |
|       net        | 52.53 | 57.16 |
|      paper       | 29.87 | 56.91 |
|     pavement     | 48.03 | 85.76 |
|      pillow      |  0.0  |  0.0  |
|   plant-other    | 27.87 | 30.27 |
|     plastic      | 16.85 | 25.25 |
|     platform     | 49.37 | 55.79 |
|   playingfield   | 60.95 | 69.98 |
|     railing      |  0.05 |  0.05 |
|     railroad     | 56.27 | 89.34 |
|      river       | 47.54 | 92.76 |
|       road       | 76.87 | 86.12 |
|       rock       | 57.34 | 81.09 |
|       roof       | 25.76 | 34.79 |
|       rug        | 58.84 | 68.68 |
|      salad       |  0.0  |  0.0  |
|       sand       | 63.62 | 79.03 |
|       sea        | 75.81 | 93.63 |
|      shelf       | 18.32 | 44.97 |
|    sky-other     | 62.14 |  88.1 |
|    skyscraper    | 57.28 | 68.92 |
|       snow       | 88.33 | 94.69 |
|   solid-other    |  0.0  |  0.0  |
|      stairs      | 46.39 | 67.51 |
|      stone       |  0.0  |  0.0  |
|      straw       | 15.59 | 33.19 |
| structural-other |  0.0  |  0.0  |
|      table       | 20.29 |  33.7 |
|       tent       | 65.86 | 82.32 |
|  textile-other   |  17.8 | 25.05 |
|      towel       | 40.82 | 42.54 |
|       tree       | 78.66 |  93.6 |
|    vegetable     |  47.9 | 68.54 |
|    wall-brick    | 52.22 | 68.49 |
|  wall-concrete   |  7.76 | 84.44 |
|    wall-other    | 10.07 | 10.59 |
|    wall-panel    |  0.0  |  0.0  |
|    wall-stone    | 40.65 |  49.9 |
|    wall-tile     | 61.45 | 94.53 |
|    wall-wood     | 41.37 | 63.97 |
|   water-other    | 14.65 | 14.94 |
|    waterdrops    |  0.0  |  0.0  |
|   window-blind   | 34.87 |  71.5 |
|   window-other   | 48.62 | 62.69 |
|       wood       | 12.66 | 21.92 |
+------------------+-------+-------+
{'aAcc': 73.17, 'mIoU': 52.0, 'mAcc': 64.1}
```

Stuff Segmentation

```
Mean IOU       @[ classes=  leaves ] = 0.3007
FW IOU         @[ classes=  leaves ] = 0.5923
Mean accuracy  @[ classes=  leaves ] = 0.4080
Pixel accuracy @[ classes=  leaves ] = 0.7328
Mean IOU       @[ classes=supercats ] = 0.5855
FW IOU         @[ classes=supercats ] = 0.7167
Mean accuracy  @[ classes=supercats ] = 0.7039
Pixel accuracy @[ classes=supercats ] = 0.8304
class | class name           |  iou   |  macc
92    | banner               | 0.2896 | 0.6410
93    | blanket              | 0.0471 | 0.0490
94    | branch               | 0.1754 | 0.1907
95    | bridge               | 0.2911 | 0.4119
96    | building-other       | 0.5348 | 0.7146
97    | bush                 | 0.2836 | 0.3592
98    | cabinet              | 0.5399 | 0.7332
99    | cage                 | 0.2267 | 0.3739
100   | cardboard            | 0.3905 | 0.5230
101   | carpet               | 0.4863 | 0.7033
102   | ceiling-other        | 0.6000 | 0.8341
103   | ceiling-tile         | 0.0000 | 0.0000
104   | cloth                | 0.0000 | 0.0000
105   | clothes              | 0.1589 | 0.1918
106   | clouds               | 0.5148 | 0.7078
107   | counter              | 0.2717 | 0.4903
108   | cupboard             | 0.0000 | 0.0000
```

109	curtain	0.6111	0.7568
110	desk-stuff	0.4120	0.5888
111	dirt	0.3920	0.6330
112	door-stuff	0.3492	0.5800
113	fence	0.3232	0.5561
114	floor-marble	0.0000	0.0000
115	floor-other	0.2041	0.2856
116	floor-stone	0.0122	0.0145
117	floor-tile	0.5990	0.7349
118	floor-wood	0.5633	0.7605
119	flower	0.4130	0.6394
120	fog	0.0529	0.0553
121	food-other	0.2344	0.2974
122	fruit	0.2396	0.3346
123	furniture-other	0.1198	0.1454
124	grass	0.6778	0.8400
125	gravel	0.2422	0.3302
126	ground-other	0.0001	0.0001
127	hill	0.1138	0.1406
128	house	0.2418	0.2892
129	leaves	0.1968	0.2316
130	light	0.3313	0.4304
131	mat	0.0000	0.0000
132	metal	0.2600	0.3511
133	mirror-stuff	0.4630	0.5993
134	moss	0.0000	0.0000
135	mountain	0.5194	0.6933
136	mud	0.0000	0.0000
137	napkin	0.0000	0.0000
138	net	0.3873	0.5665
139	paper	0.2706	0.3797
140	pavement	0.4802	0.6560
141	pillow	0.0004	0.0004
142	plant-other	0.1192	0.1766
143	plastic	0.0867	0.1036
144	platform	0.2613	0.4320
145	playingfield	0.6762	0.8582
146	railing	0.0000	0.0000
147	railroad	0.5888	0.8049
148	river	0.4772	0.7329
149	road	0.6336	0.8534
150	rock	0.4942	0.7822
151	roof	0.1374	0.1623
152	rug	0.3178	0.4712
153	salad	0.0000	0.0000
154	sand	0.5991	0.6658
155	sea	0.8463	0.9309
156	shelf	0.2912	0.4477
157	sky-other	0.7164	0.8574
158	skyscraper	0.3484	0.4586
159	snow	0.8814	0.9418
160	solid-other	0.0000	0.0000
161	stairs	0.1998	0.3488
162	stone	0.0004	0.0004
163	straw	0.2523	0.3538
164	structural-other	0.0000	0.0000
165	table	0.2199	0.3277
166	tent	0.0801	0.1002
167	textile-other	0.1107	0.1653
168	towel	0.2633	0.3497
169	tree	0.7186	0.8939
170	vegetable	0.2972	0.3957
171	wall-brick	0.4457	0.6510
172	wall-concrete	0.5664	0.7785
173	wall-other	0.1750	0.2648
174	wall-panel	0.0000	0.0000
175	wall-stone	0.2892	0.3530
176	wall-tile	0.6257	0.8190
177	wall-wood	0.3684	0.5274
178	water-other	0.2423	0.3221
179	waterdrops	0.0000	0.0000
180	window-blind	0.4380	0.5565
181	window-other	0.4135	0.6813
182	wood	0.1498	0.2071
183	other	0.8158	0.9459

Panoptic Segmentation

```
          |    PQ     SQ     RQ     N
---------------------------------------
All       |  44.2   81.3   52.8   133
Things    |  53.0   83.8   62.6    80
Stuff     |  30.9   77.4   38.1    53
```

Image Captioning

```
Bleu_1: 0.783
Bleu_2: 0.638
Bleu_3: 0.500
Bleu_4: 0.387
METEOR: 0.279
ROUGE_L: 0.584
CIDEr: 1.221
SPICE: 0.216
```

▶ **Eval COCO-Stuff 10K**

Download the evaluation code from:

- eval_10k Extract the zip file and put the `eval_10k` folder into `Project Root` folder. The folder hierarchy should be:

```
/
├─ data/
├─ model/
├─ utils/
├─ eval_10k/
|   ├─ test_stuff_10k/
|   |   └─ *.jpg
|   ├─ cocostuff-10k-v1.1.json
|   ├─ eval_stuff_10k.py
|   ├─ predict_10k.py
|   └─ test_stuff_10k.txt
etc.
```

Files in this zip:

- test_stuff_10k folder: stuff 10k test set (1k images)
- cocostuff-10k-v1.1.json: ground truth file of stuff 10k
- eval_stuff_10k.py: evaluation code for stuff 10k
- predict_10k.py: generate the json file of predictions
- test_stuff_10k.txt: file list of stuff 10k test set

Usage:

1. Use the following command to create prediction file:

```
python eval_10k/predict_10k.py --weights /path/to/weights.pt --source eval_10k/test_stuff_10k --data
/path/to/coco.yaml --img 640 --device 0 --export-mask --color-map /path/to/color_map.pickle
```

The prediction file `predictions_semantic.json` will be created under the predict result folder (default: `runs/predict-sem/exp{n}/`)

2. Then, use the following command for evaluation:

```
python eval_stuff_10k.py --gt eval_10k/cocostuff-10k-v1.1.json --pred /path/to/predictions_semantic.json
```

Training

Generalist Visual Language Tasks

Use DDP for training:

```
python -m torch.distributed.launch --nproc_per_node 8 --master_port 9527 caption/train.py --data coco.yaml --epochs 60
--batch 64 --img 640 --cfg models/caption/gyolo.yaml --name gyolo --weights '' --hyp data/hyps/hyp.scratch-cap.yaml --
device 0,1,2,3,4,5,6,7 --optimizer AdamW --flat-cos-lr --no-overlap --close-mosaic 2 --save-period 1 --noval --noplots
```

Cider Optimization for Improving Image Captioning:

```
python -m torch.distributed.launch --nproc_per_node 8 --master_port 9527 caption/tune.py --weights 'gyolo.pt' --cfg
models/caption/gyolo.yaml --name gyolo-co --data data/coco.yaml --hyp data/hyps/hyp.scratch-cap.yaml --epochs 10 --
batch-size 64 --img 640 --device 0,1,2,3,4,5,6,7 --optimizer AdamW --save-period 1
```

Use torchrun for distributed training on slurm cluster:

```
torchrun --nproc_per_node=8 --nnodes=2 --node_rank=$NODE_RANK --master_addr=$MASTER_ADDR --master_port=9527
caption/train.py --device 0,1,2,3,4,5,6,7 --batch 128 --epochs 40 --workers 16 --data data/coco.yaml --img 640 --cfg
models/caption/gyolo.yaml --name gyolo --weights '' --hyp data/hyps/hyp.scratch-cap.yaml --optimizer AdamW --flat-cos-
lr --no-overlap --close-mosaic 2 --save-period 1 --noplots
```

Cider Optimization for Improving Image Captioning:

```
torchrun --nproc_per_node=8 --nnodes=2 --node_rank=$NODE_RANK --master_addr=$MASTER_ADDR --master_port=9527
caption/tune.py --weights 'gyolo.pt' --cfg models/caption/gyolo.yaml --name gyolo-co --data data/coco.yaml --hyp
data/hyps/hyp.scratch-cap.yaml --epochs 10 --batch-size 128 --img 640 --device 0,1,2,3,4,5,6,7 --optimizer AdamW --
save-period 1
```

Note: Please replace `$MASTER_ADDR` and `$NODE_RANK` to your own machine.

Inference

```
python caption/predict.py --weights 'gyolo.pt' --source $SOURCE --data data/coco.yaml --img 640 --device 0 --export-
mask
```

Note: Please replace `$SOURCE` with the image/video you want to infer.

Citation

```
@inproceedings{wang2024yolov9,
  title={{Generalist YOLO}: Towards Real-Time End-to-End Multi-Task Visual Language Models},
  author={Chang, Hung-Shuo and Wang, Chien-Yao and Wang, Richard and Chou, Gene and Liao, Hong-Yuan Mark},
  booktitle={IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)},
  year={2025},
  pages={6217-6227}
}
```

Coming Soon

▶ **Coming soon**

- Visual Grounding
- Whole Body Pose Estimation
- Visual Question Answering
- Video Understanding Tasks
- Specific Vision Tasks

Contributors

▶ **Contributors**

- @ws6125 : Most of generic vision tasks and visual language tasks.
- @110598074 : Most of specific vision tasks.
- @F84064014 : Video panoptic segmentation.
- @k22708038 : Visual grounding.
- @HuChengXue : Whole body pose estimation.
- @waue0920 : Multiple node training.
- ...

Acknowledgements

▶ **Reference Codes**

- https://github.com/WongKinYiu/yolov9
- https://github.com/davidnvq/grit

- https://github.com/djiajunustc/TransVG
- https://github.com/Robertwyq/PanoOcc
- https://github.com/junjiehe96/FastInst
- https://github.com/fan23j/yolov7-pose-whole-body
- ...

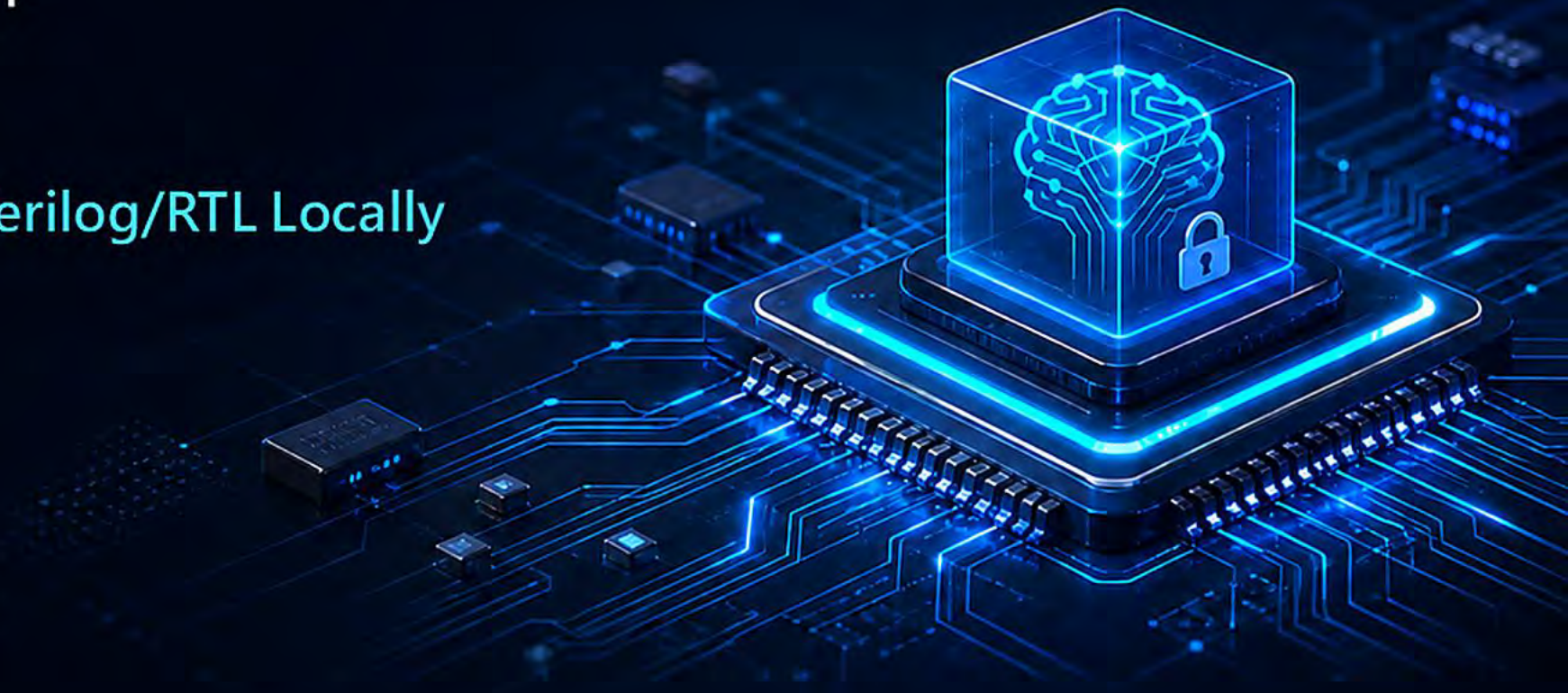
SiliconMind

SiliconMind-V1

SiliconMind-V1

AI Engine for IC Design

Generate · Test · Debug Verilog/RTL Locally



Academia Sinica & NTU

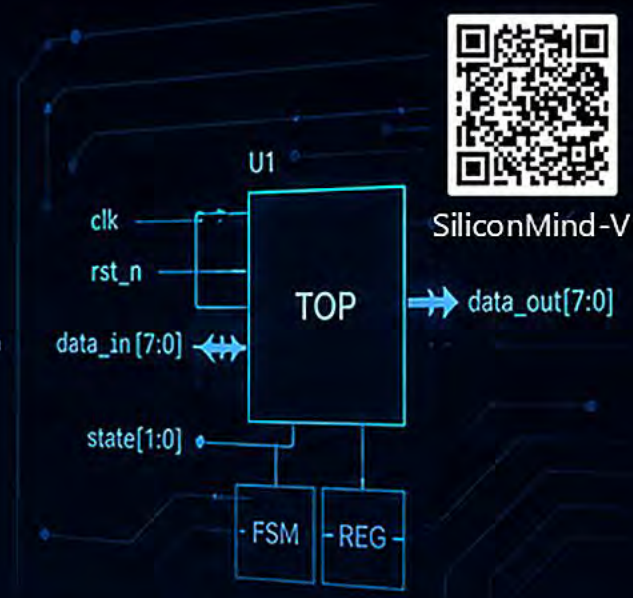
SiliconMind

Why IC Design Needs Private AI

Keep your design data and silicon secure

```
module top (
    input clk,
    input rst_n,
    input [7:0] data_in,
    output reg [7:0] data_out;

    // FSM control
    always @(pesedge clk or negedge rst_n) begin
        if (!rst_n)
            state <= S_IDLE;
        else
            state <= next_state;
    end
endmodule
```



SiliconMind-V1

Sensitive IP

RTL code and IP libraries are confidential

Cloud Risk

Cloud LLMs may expose source code and inference logs

Correctness Matters

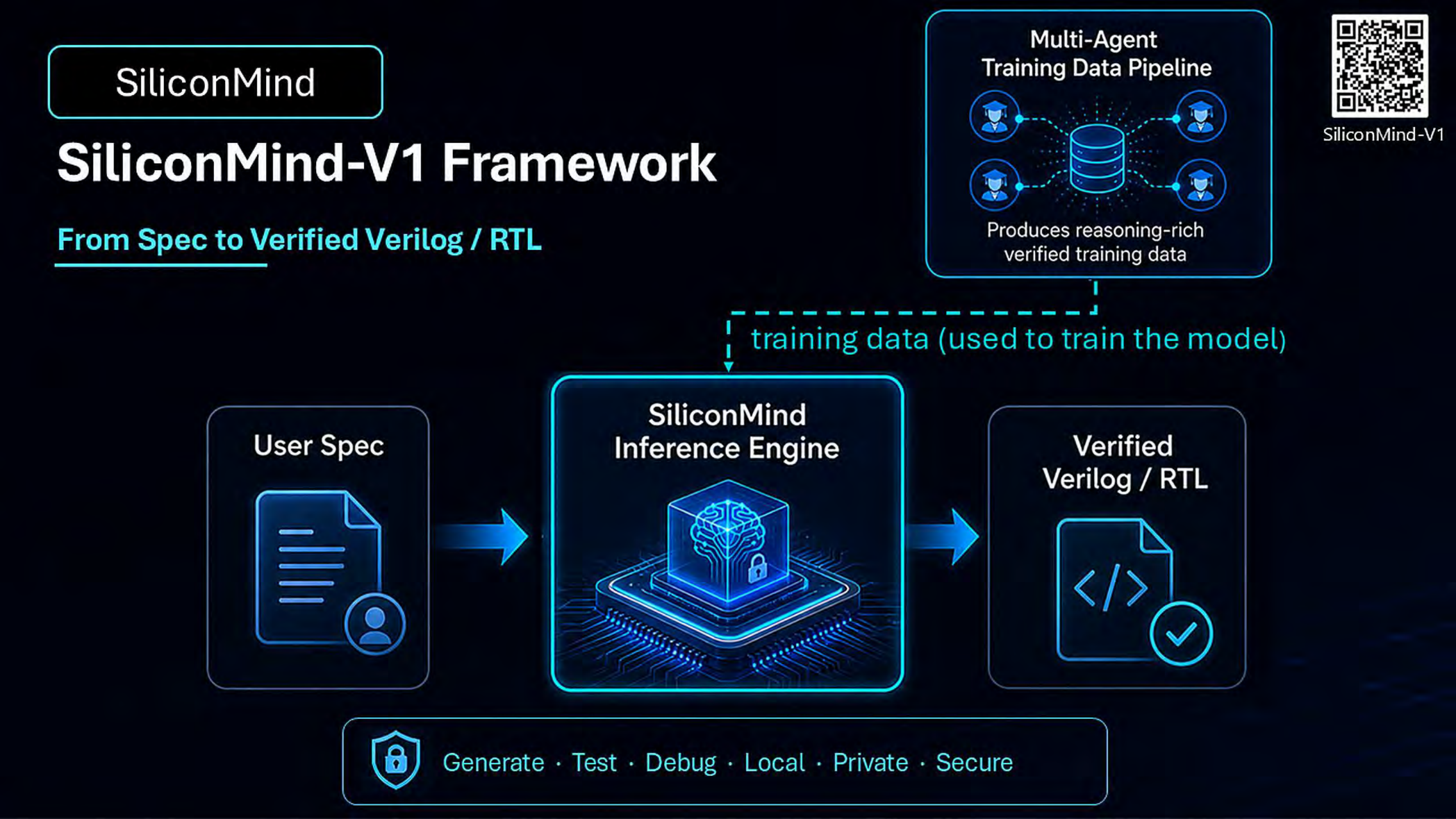
Syntax-correct Verilog is not enough

Generate · Test · Debug Verilog/RTL Locally

SiliconMind

SiliconMind-V1 Framework

From Spec to Verified Verilog / RTL



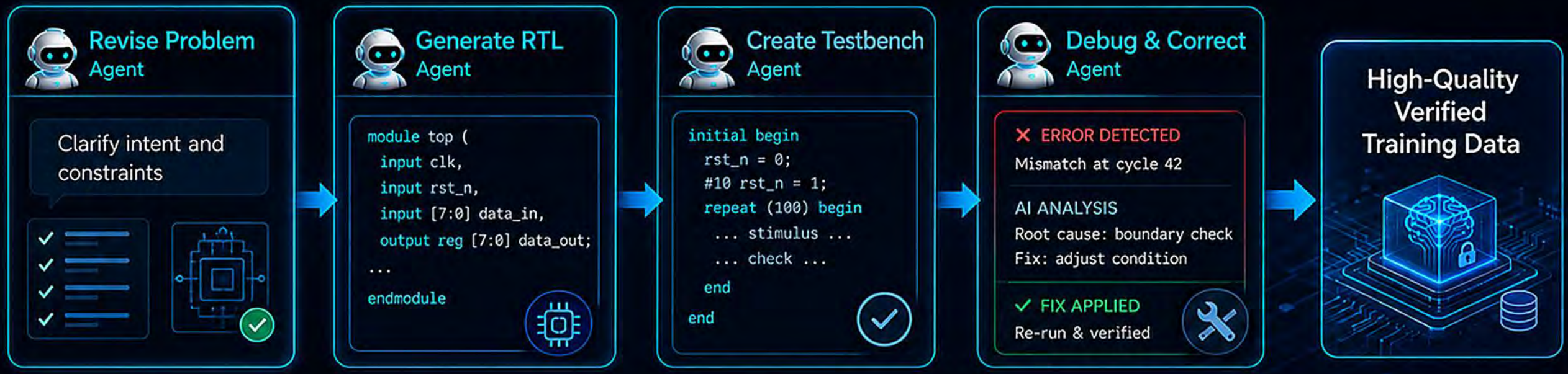
SiliconMind-V1

SiliconMind

SiliconMind-V1

Multi-Agent Distillation Pipeline

Teach the model to generate, test, and fix



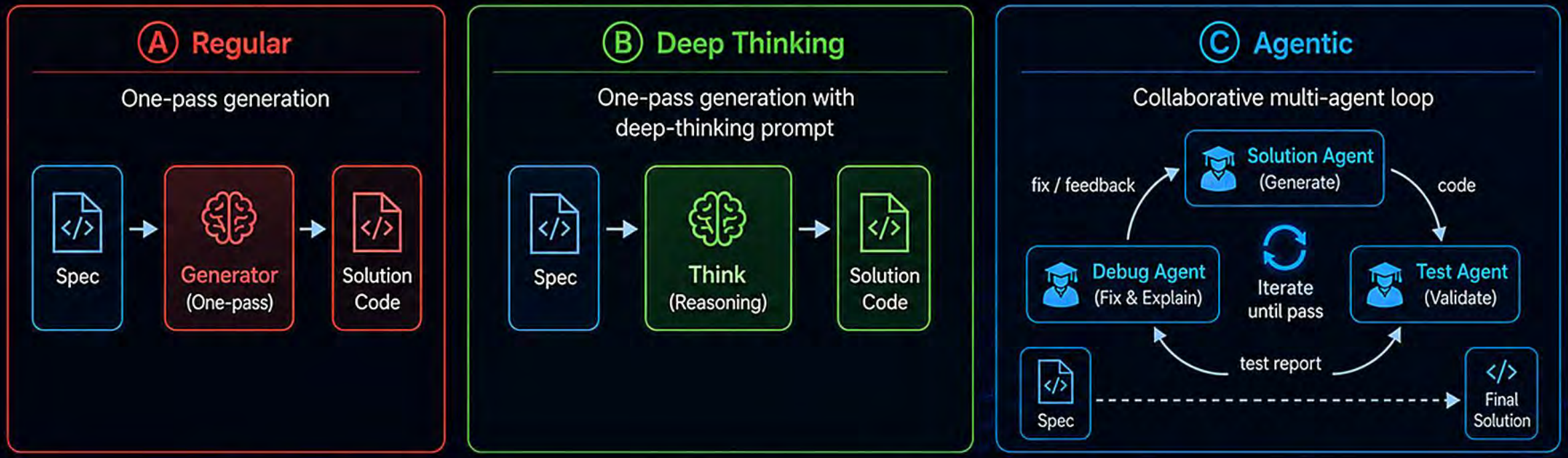
GENERATE TEST DEBUG

SiliconMind

SiliconMind-V1

From Code Generation to Debug-Reasoning

Generate, test, and debug with test-time scaling



Higher Accuracy
Fewer bugs, better results

Test-Time Scaling
More thinking, better code

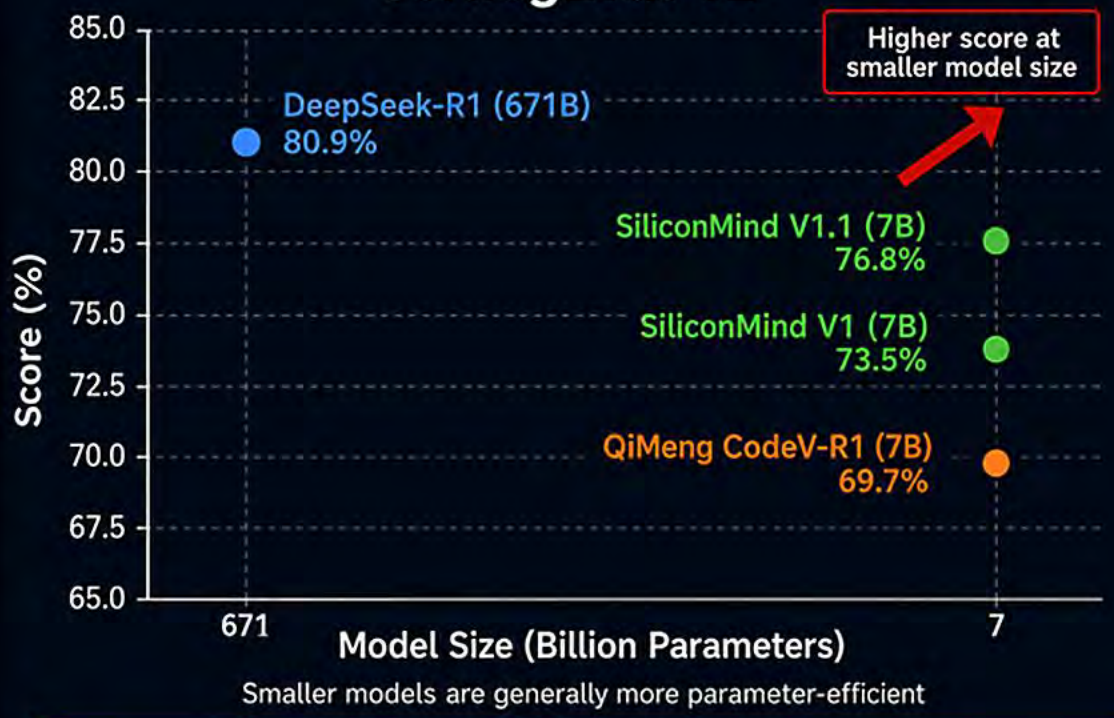
Robust & Reliable
Self-debug reduces errors

Faster to Deploy
Less human-in-the-loop

SiliconMind

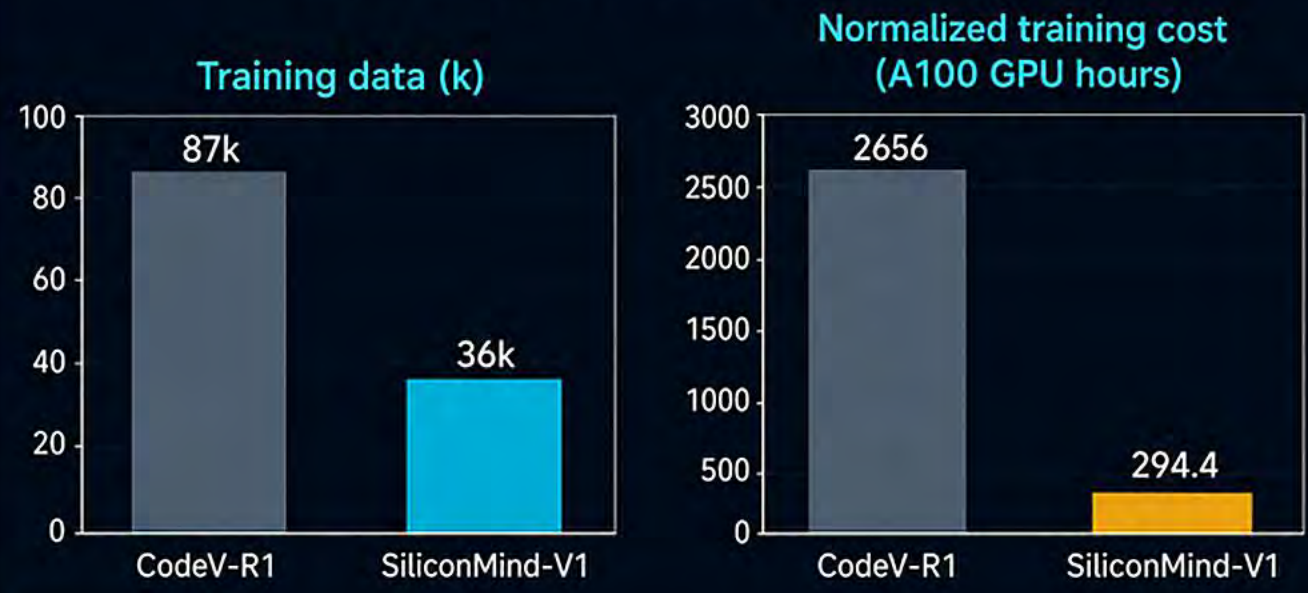
SiliconMind-V1

VerilogEval-v2



Smaller models are generally more parameter-efficient

Lightweight training pipeline



Surpassing Previous SOTA

SiliconMind-V1 outperforms QiMeng-CodeV-R1, a model released by the Chinese Academy of Sciences, across multiple benchmarks, while narrowing the gap with DeepSeek-R1, which has over **95×** more parameters.

High Training Efficiency

Compared with QiMeng-CodeV-R1, SiliconMind-V1 uses only **50%** of the training data and **1/9** of the compute while achieving comparable or better results.

Taiwan Tongues

Building Language Resources for Taiwan in the AI Era

English and Simplified Chinese Dominate AI Training Data

Taiwan's Languages Remain Underrepresented in AI Training Data

Limited Local Data Limits AI Growth

Why It Matters

- Most LLMs are trained primarily on English and Simplified Chinese datasets.
- Taiwan’s languages and cultural context remain underrepresented in AI systems.
- Without sufficient local data, AI may generate translations or responses that sound unnatural or fail to meet Taiwan’s needs.

LEARN MORE

What We Do

- Build a General Corpus of Taiwan's Languages to support AI research and model training.
- Collect high-quality, legally licensed language resources from Taiwan.

1 Writer-Authorized Texts

2 Wikipedia Translation Corpus

Our Outcome

SARC-Taigi-LLM is a Taigi large language model trained on writer-authorized Taigi corpus data.

Jointly developed by the AI Research Center at National Yang Ming Chiao Tung University and IMA.

Demonstrates how local language resources can help AI understand and generate Taiwanese content.

Join Us

Apply for access to Taiwan Tongues datasets

Contribute and authorize language resources

Explore our datasets on Hugging Face

1 Apply for Data Access

2 Data Contribution & Authorization

3 Hugging Face Datasets

IMA 資訊經理人協會 Information Management Association

Twinkle AI

開源社群
Discord

> Twinkle AI 開源語言模型社群的核心討論場域。在這裡與開發者、研究員及開源愛好者交流，共同打造台灣的 AI 技術生態。

加入 Discord 社群

Twinkle AI

PORTAL 02 / OFFICIAL WEBSITE

官方網站
Twinkle AI

全方位的專案資訊中心。獲取最新公開的語言模型資訊、社群活動訊息以及開源發展的完整藍圖。

瀏覽官方網站

Twinkle AI

Twinkle Hub

台灣資料專屬的 MCP (Model Context Protocol) 服務。優化 AI 模型與在地化資料的連結效率，提供更精準、具備文化脈絡的智慧應用程式。

啟動 MCP 服務

Twinkle AI

GitHub
開源原始碼

收錄教學資源、Twinkle Eval 評分工具與 Leaderboard。開放透明的技術根基，共同創造底層架構。

查看開源專案

Twinkle AI

Hugging Face

Twinkle AI 的模型與資料集託管中心。在這裡您可以即刻獲取正體中文預訓練模型，並開始進行測試、微調或各種商業應用部署。

模型與資料集

Twinkle AI

Discord Community

> The heartbeat of the Twinkle AI community. Join our Discord to connect with developers, researchers, and open-source enthusiasts as we build Taiwan's premier AI ecosystem.

Join Our Discord

Twinkle AI

Official Website

> Your central hub for everything Twinkle AI. Dive into our latest LLM releases, technical documentation, upcoming community events, and our open-source roadmap.

Visit Official Website

Twinkle AI

Twinkle Hub

A specialized Model Context Protocol (MCP) service built for Taiwanese data. Seamlessly integrate localized context into your AI models for highly precise, culturally aware applications.

Launch MCP Service

Twinkle AI

GitHub Codebase

> The foundation of our open-source mission. All of our codebases and toolkits are hosted here. Collaboration hub featuring tutorials, Twinkle Eval tools, and Leaderboards for developers.

Contribute on GitHub

Twinkle AI

Hugging Face

Our central repository for models and datasets. Easily download, test, and deploy high-quality Traditional Chinese LLMs trained by the Twinkle AI community.

Access AI Models

源來適你

OpenSource4You

since 2022

From Taiwan to the World — Open Source Community

Who Are We?

A Loose Organization, Serious About Open Source

OpenSource4You is a loosely organized but deeply passionate open source community. We bring together engineers, students, and industry professionals who care about open source. Our goal is to "let the community empower the individual" — through real contributions to open source projects, we help every member shine on the global stage.

large-project
20+
maintainers cultivated

every month
40+
online events

every month
20,000+
technical discussions

every year
30 +
in-person events

What We Do

Mentorship Program

Senior contributors guide new members one-on-one, helping them contribute to real international open source projects from the ground up. No matter your current level, you can progress step by step toward becoming a core open source developer.

Technical Activities

- "Tech Talks" — online technical sharing sessions
- Study groups on Kafka, Ray, Flyte, and more
- In-person meetups

Content Production

- Facebook community posts
- GitHub technical notes
- "OpenSource4You" podcast

International Engagement

- Technical talks at international conferences
- Taiwan's first Apache Software Foundation chapter

Apache Kafka

The Undefeated Legend of the Modern Data World

Kafka is the absolute standard for distributed event streaming worldwide. Over 80% of Fortune 100 companies rely on it to run their core infrastructure; Netflix, Uber, and LinkedIn process trillions of real-time messages with it daily — rock-solid and never down.

- Million-messages-per-second throughput with industry-leading low latency
- Highly reliable architecture with zero data loss
- KRaft architecture upgrade makes cluster management easier

**When the world's engineers face extreme concurrency,
this is their one shared faith.**

Apache Airflow

The Air Traffic Control Tower for Data Pipelines

Apache Airflow is one of the most widely used workflow scheduling and monitoring platforms in the world. Engineers define pipelines in Python as a first-class citizen, scheduling, monitoring, and managing complex cross-system workflows with ease. From daily reporting to ML training, Airflow handles it end-to-end.

- Python-first workflow definitions, minimal learning curve for engineers
- Rich operator ecosystem, integrating hundreds of databases and cloud services
- AI-native integration with smart, agent-in-the-loop retries.
- Full-featured web UI with clear visibility into pipeline status.

**An essential weapon for data engineers
— the industry standard, no question.**

RAY

The Compute Brain of the Big AI Era

Ray is the next-generation operating system for AI infrastructure. OpenAI uses it to orchestrate tens of thousands of GPUs to train GPT-4; with just one Python decorator, @ray.remote, you can scale a program from your laptop into a distributed system commanding thousands of servers. NVIDIA, AWS, and Google Cloud all race to integrate it natively — it sits at the core of the entire AI ecosystem.

- Distributed scaling with a single command — developer-friendly
- The industry's first choice for large-scale model training and inference
- Tech giants compete to integrate it as the ecosystem keeps growing

The compute faith of every AI engineer.

Apache Ozone

The Infinite Warehouse of the Cloud-Native Era

Ozone is a distributed object storage system built for massive-scale deployments — the next-generation storage core of the Hadoop ecosystem. It breaks through the bottleneck traditional HDFS faces with huge numbers of small files, scaling effortlessly to billions of objects and hundreds of petabytes in cloud-native Kubernetes environments, all while maintaining extremely high throughput and availability.

- Supports billions of objects, fully solving small-file performance issues
- Kubernetes-native, with seamless hybrid cloud and on-premises deployment
- S3 API compatible — existing tools plug in with zero changes

Letting enterprise data lakes truly become boundless and bottomless.

Flyte

The Workflow Engine That Precisely Commands Billions of Tasks

Open-sourced by Lyft, Flyte is a Kubernetes-native workflow engine purpose-built for machine learning and data pipelines. Top companies like Spotify use it to precisely orchestrate complex AI workflows. Its intelligent caching mechanism lets tasks resume directly from the point of interruption, saving substantial recomputation costs.

- Strongly typed design with complete, traceable data lineage
- GitOps integration — pipeline versions are controllable with one-click rollback
- Kubernetes-native architecture, born for the cloud

Freeing ML engineers from all-night overtime.

Apache DataFusion

The Query-Speed Legend Built in Rust

DataFusion is a high-performance embeddable query engine built in Rust — a core component of the Apache Arrow ecosystem. Its vectorized execution engine squeezes every ounce of power from modern CPUs, making SQL queries astonishingly fast. Next-generation data platforms like InfluxDB IOx and Delta Lake use DataFusion as their underlying query brain.

- Native Rust implementation, far outperforming traditional JVM query engines
- Vectorized execution paired with the Arrow memory format for ultimate IO efficiency
- Embeddable into any system — the best foundation for building custom query engines

The shared foundation of next-generation data tools, quietly rewriting the industry's rules.

Apache YuniKorn

The Intelligent Scheduling Brain of Kubernetes Clusters

YuniKorn is a next-generation Kubernetes resource scheduler designed for large-scale AI/ML workloads and multi-tenant enterprise environments. It replaces the limitations of the default scheduler with hierarchical queue management, maximizing cluster resource utilization and eliminating idle compute waste entirely.

- Gang Scheduling support ensures distributed training tasks launch as a complete batch
- Hierarchical multi-tenant queues fairly allocate cluster resources — no more scrambling
- Seamless integration with native Kubernetes APIs — zero changes to existing workflows

Letting every GPU and CPU do the right thing at the right time.

Apache Mahout

Building High-Performance Environments for Quantum Machine Learning

Mahout provides a cutting-edge environment for building scalable, performant machine learning applications, with its latest evolution focusing entirely on the frontier of quantum computing. Powered by a high-performance Rust and CUDA backend, Mahout seamlessly bridges classical data science and quantum workflows, allowing developers to write quantum applications once and execute them anywhere.

- Providing unified Python API to run circuits across Qiskit, Cirq, and Amazon Braket.
- Ultra-fast, GPU-accelerated encoding of classical data into quantum states.
- Built-in DLPack support enabling zero-copy tensor transfers.

When machine learning meets quantum computing,
Mahout is writing the next chapter.

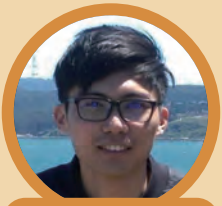
The Cornerstones of the Open Source World

Chia-Ping Tsai
Apache Member

Wei-Chiu Chuang
Apache Member

PoAn Yang
Apache Kafka
Committer

Kai-Hsun Chen
RAY
Maintainer



Kuan-Po Tseng
Apache YuniKorn
Committer

Tingyao Huang
Apache YuniKorn
PMC Member

Chung-En Lee
Apache Ozone
PMC Member

Wei Lee
Apache Airflow
PMC Member

Kevin Su
Flyte
Maintainer

TengYao Chi
Apache Kafka
Committer

Eric Chang
Apache Gravitino
Committer

Zhe-You Liu
Apache Airflow
Committer

Jie-Kai Chang
Apache Mahout
PMC Member

Chu Cheng Li
Apache Ozone
Committer

Jheng-Sing Chen
Apache Ozone
Committer

Kuan-Hao Huang
Apache Mahout
Committer

Hsien-Cheng Huang
Apache Mahout
Committer

Guan-Ming Chiu
Apache Mahout
PMC Member

Han-Ju Chen
RAY
Maintainer

Nary Yeh
Flyte
Maintainer

Jiunn-Yang Huang
Apache Kafka
Contributor

Yu-Lin Chen
Apache YuniKorn
Committer

Huang-Hsiang Cheng
Apache DataFusion
Comet Contributor

Jui-An Huang
RAY
Maintainer

Alex Wu
Flyte
Maintainer

Yu-Chen Lai
Apache DataFusion
Comet Contributor

Ming-Yen Chung
Apache Kafka
Contributor

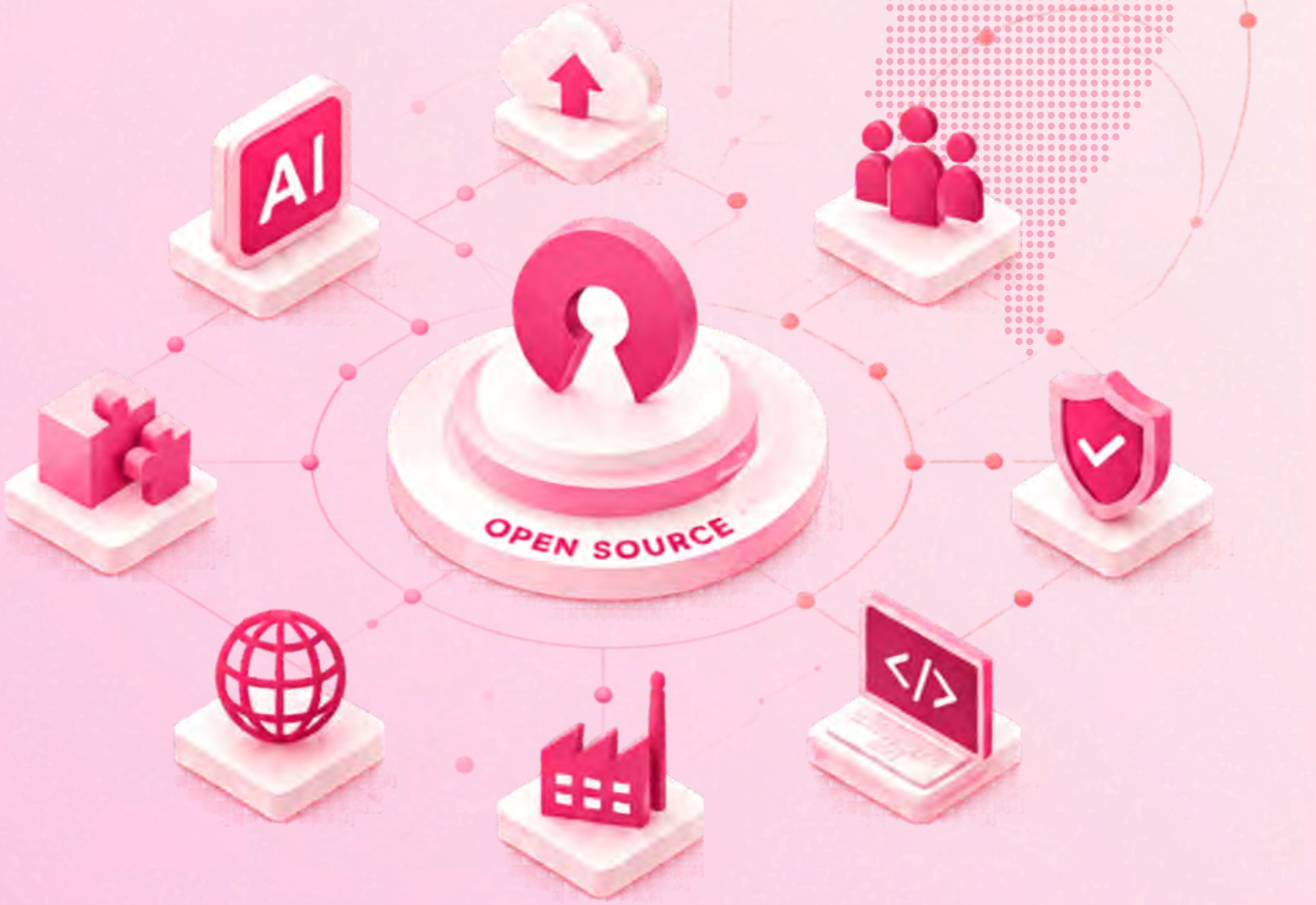
You-Cheng Lin
RAY
Maintainer

Taiwan Open Source Starts Here

InnoVEX 2026 | ADI × Open Source Team Taiwan

Administration for Digital Industries(ADI),
Ministry of Digital Affairs (moda)

A Key Driver of Taiwan's Open-Source AI Ecosystem



數位發展部 數位產業署
Administration for Digital Industries, Ministry of Digital Affairs

Open Source Team Taiwan

Click to see more open source tools

Hugging Face

GitHub

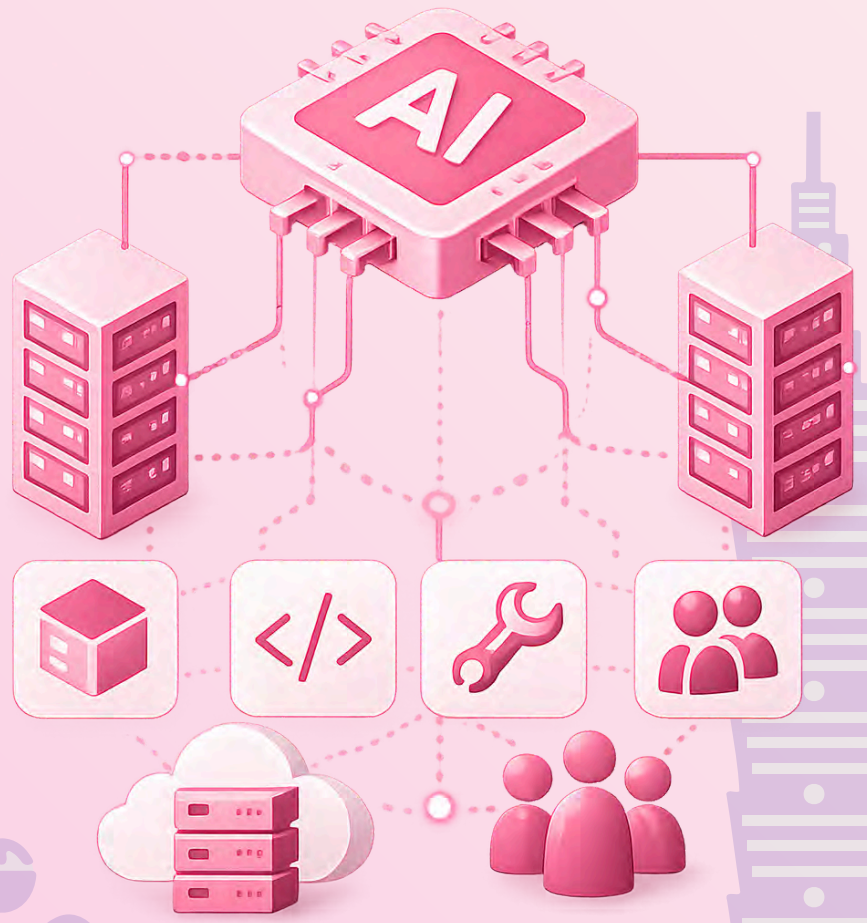
Why Open Source?

In the AI era, industrial competitiveness extends far beyond chips, servers, and hardware supply chains. It now depends on AI models, system frameworks, application tools, developer communities, and software agility.

> **Open source is shaping the universal language for global AI innovation, driving the democratization of technology and industry collaboration.**

For Taiwan,

Promoting open source is not merely about sharing code. It is about building a shared, governed, and sustainable digital infrastructure—lowering the barriers to AI adoption and amplifying Taiwan's impact in the global ecosystem.



The Role of ADI

ADI is addressing industry challenges by fostering an Open-Source AI Ecosystem.

Our core philosophy is not for the government to develop end products directly, but to leverage the open-source model to enable the industry through three key pillars:

1

Reusable AI Models

Expanding government-backed R&D achievements into diverse industry applications for broader adoption.

2

Integrable Frameworks

Providing rapid-deployment frameworks to lower technical barriers and minimize redundant development costs for enterprises.

3

Evolving Common Capabilities

Transitioning technology beyond static **project deliverables** into dynamic assets that continuously grow alongside community and industry needs.

Today, we take a major step forward in creating an

Open-Source AI Ecosystem

Key Achievements 2025

In 2025, ADI began releasing Taiwan’s key AI innovations on global platforms such as GitHub and Hugging Face. This milestone has enabled government-sponsored AI projects to transition from closed-source to the global open-source ecosystem.

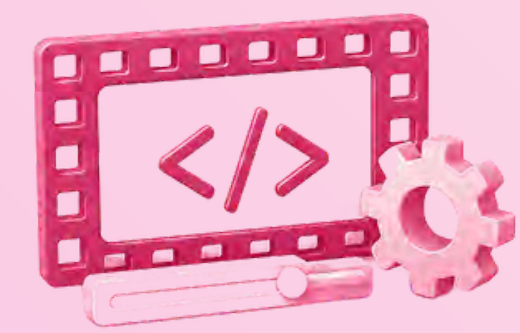
Featured Open-Source Innovations

Taiwan Tongues ASR CE

Empowering voice technology development tailored for Taiwan’ s multi-lingual and localized environments.

MedAIgent

Exploring and optimizing the integration of AI into clinical workflows and patient care processes.



AI Smart Recode

Enhancing efficiency in media processing, optimization, and downstream video content applications.

Digital Reimbursement Service

Driving process digitization, regulatory compliance, and operational efficiency in financial workflows.

Click to see more open source tools

Hugging Face

GitHub

These achievements signify Taiwan's pivotal shift: transforming isolated AI projects into visible, accessible, and highly adaptable open-source assets for global co-innovation.

數位發展部 數位產業署
Administration for Digital Industries, moda

From Projects to an Open-Source Ecosystem

The value of open source lies not only in making source code public, but in fostering long-term collaboration among enterprises, communities, developers, and the government. In 2025, ADI supported numerous expert-led open-source summits that brought together enterprise CIOs, community leaders, engineers, and policymakers to tackle key challenges: barriers to open-source adoption, corporate governance, participation models, commercialization strategies, and global trends.

In 2025, ADI gradually expanded its role from a technology promoter to an ecosystem coordinator. In addition to promoting open source technology, Taiwan has also begun to establish

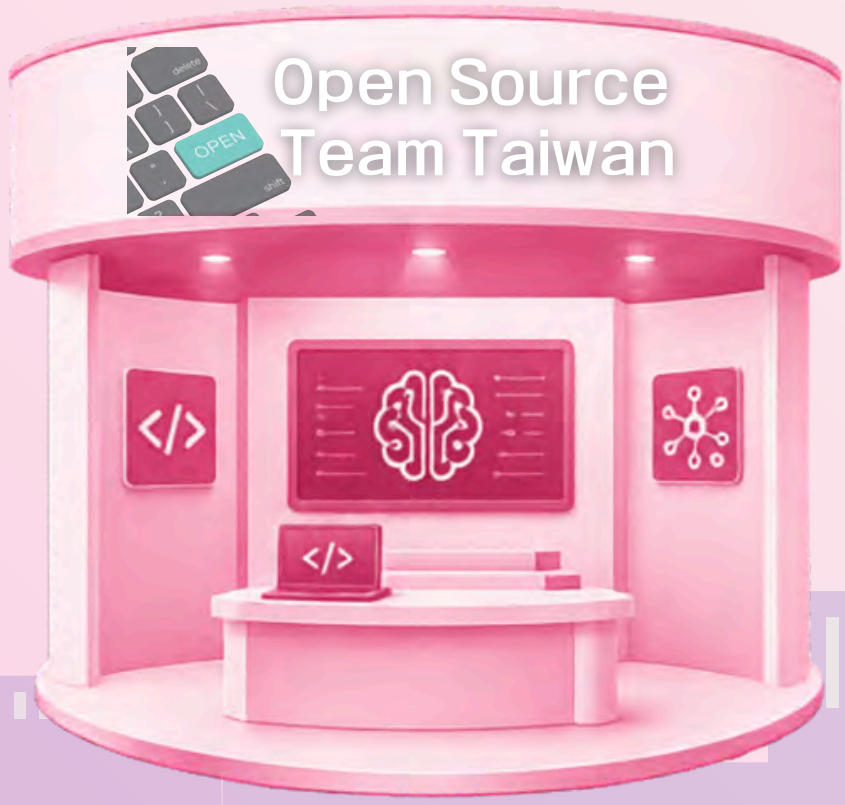
- Taiwanese open source governance culture
- Industry collaboration models
- AI technology self-reliance capabilities
- International open source connections

Our vision for an interconnected AI value chain

Models » Systems » Applications » Real-world Scenarios

From **Projects**
to an **Open-Source Ecosystem**

At the Open Source Team Taiwan pavilion, we demonstrate that Taiwan is more than a hardware powerhouse. We are building a vibrant AI open-source ecosystem.



ADI × Open Source Team Taiwan

Empowering Taiwan to evolve from AI users to global AI contributors and co-creators in the AI ecosystem.

InnoVEX 2026 | ADI × Open Source Team Taiwan

數位產業署
臺灣AI產業開源的重要推手

Building Taiwan's Open Source AI Ecosystem

數位發展部 數位產業署
Administration for Digital Industries, Ministry of Digital Affairs

Open Source Team Taiwan

點擊查看更多開源工具

Hugging Face

GitHub

為什麼是開源？

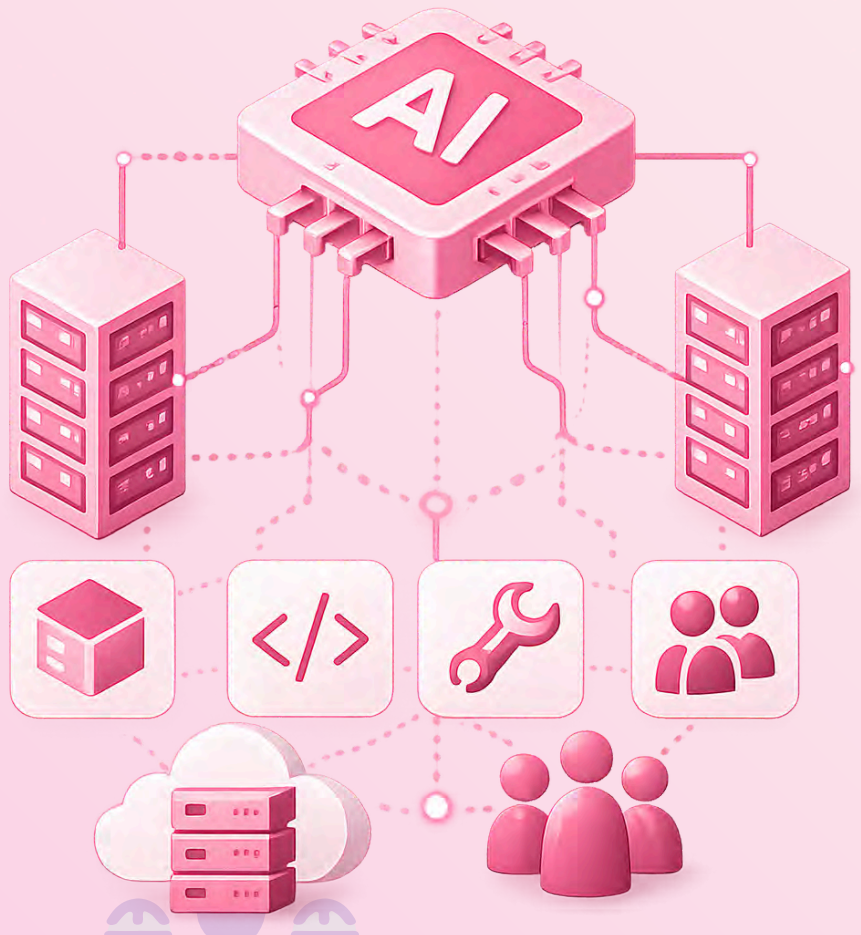
在 AI 時代，產業競爭力除了來自晶片、伺服器與硬體供應鏈，也包含 AI 模型、系統框架、應用工具、開發社群與持續演進的軟體能力。

全球 AI 發展正快速建立在開源基礎之上。
開源正成為全球 AI 創新的共同語言，除了開放共享技術資源，更促進跨產業協作，加速創新成果落地應用。

對臺灣而言，

推動產業開源的核心，是建立可共用、可治理、可持續演進的數位基礎能力。

我們透過開源，公開程式碼，讓更多企業能以更低門檻導入 AI，並在全球開源生態中被看見。



數位產業署的角色

數位產業署的核心思維並非由政府直接發展終端產品

我們以**開源 AI 生態系**作為重要推動方向。

1 建立可重複使用的 AI 模型

讓政府支持的研發成果能被更多產業場域延伸應用

2 建立可快速整合的系統框架

降低企業導入 AI 的技術門檻與重複開發成本

3 建立可持續演進的共通能力

讓技術成果在完成專案驗收後，仍能隨著社群參與及產業需求持續演進與成長。

這 些 能 力 共 同 形 成

Open-Source AI Ecosystem
開源 AI 生態系

2025年的重要推動成果

2025 年，數位產業署開始透過 GitHub、Hugging Face 等國際平台，陸續公開臺灣 AI 領域的重要開源成果，讓政府支持的 AI 技術不再停留於封閉專案，使政府支持的 AI 技術成果逐步走出封閉專案框架，進入全球開源生態。

實證案例成果：

Taiwan Tongues ASR CE

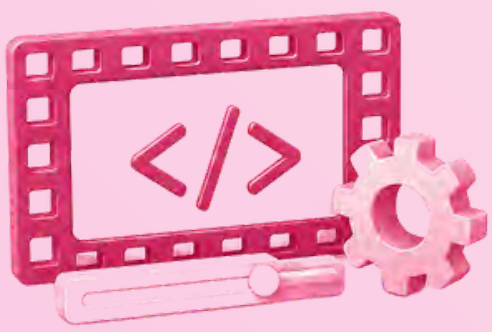
開源語音辨識模型-支援臺灣多語言混合辨識

提供200小時多語語料庫及2小時標註測試集
釋出模型權重並支援本地端環境部署
供開放可驗證的技術基盤，降低企業導入門檻

MedAIgent

AI醫護虛實共照 藥事Agent-即時用藥風險分析

提供用藥處方決策警示與替代建議
有效辨識藥物不良反應，提升醫生判讀準確性
採用GPT-OSS 120B並導入嚴謹安全訓練



AI Smart Recode

AI智慧轉碼-為影音內容進行有效率的轉碼與應用

建立系統化輔助工具，加快程式碼轉換與驗證流程
自動生成系統分析文件，快速掌握業務邏輯
運用大語言模型解析老舊程式碼，提升精準度

Digital Reimbursement Service

數位核銷服務網-建構票據辨識與自動化報銷系統

基於FastAPI框架開發，具備高性能與非同步處理能力
模型支援兼顧隱私與解析效能
自動化解析收據與發票影像

點擊探索更多開源成果

Hugging Face

GitHub

這些成果代表，臺灣的 AI 能力正在從單一專案，走向可被看見、可被使用、可被再創新的開源資源。

數位發展部 數位產業署
Administration for Digital Industries, moda

InnoVEX 2026 | ADI × Open Source Team Taiwan

從 開源成果 到 開源生態系

開源的價值，不只在於成果公開，更在於形成企業、社群、工程師與政府之間的長期協作關係。數位產業署在 2025 年支持多場開源專家會議，邀集企業 CIO、開源社群代表、工程師與政府單位，共同討論企業導入開源的困難、開源治理、企業參與模式、開源商業化與國際趨勢。

這代表數位產業署的角色正在轉變，從技術提供者，走向生態系協調者

除了推動技術開源，也開始建立：

- 臺灣開源治理文化
- 產業協作模式
- AI 技術自主能力
- 國際開源連結

目標是逐步形成完整 AI 產業鏈：

模型 >> 系統 >> 應用 >> 場域

從 開源成果
到 開源生態系

在 Open Source Team Taiwan 展館中，
我們希望向世界展示：
臺灣不僅具備完整的 AI 硬體供應鏈，也正在建立
自己的 AI 開源生態系。

ADI × Open Source Team Taiwan

在數位產業署的推動下，臺灣產業將從 AI 技術應用出發，進一步參與全球 AI 生態系，成為技術發展的貢獻者與共同創造者。

Let’s Stay in Touch

Scan the QR code to leave your contact information.

Contact Us:

Information Management Association (IMA),Taiwan

dginfra@ima.org.tw

https://www.ima.org.tw/